Statistiques Fonctionnelles

CARLU Jonas, AFANE Fouad, LANCON Cindy

2026-02-24

1 - Introduction

Ce projet a pour objectif d'étudier les profils de consommation électrique de $n = 6\,291$ entreprises, observées sur $D = 672$ instants de mesure espacés de 30 minutes sur deux semaines consécutives. On note i = 1,. . . ,n, l'indice d'une entreprise et $t \in [1, 672]$ l'instant de mesure.

Les données présentent plusieurs propriétés qui justifient une approche par statistiques fonctionnelles :

- **Continuité** : les profils de consommations observés $Y_i(t)$ sont des discrétisations d'une fonction continue sous-jacente.
- **Ordre** : l'intervalle temporel [1,672] est ordonnée et muni d'une distance : l'instant t_{10} arrive necessairement après t_9, ce qui distinque les données fonctionnelles d'un simple vecteur de variables.
- **Cohérence** : les courbes sont toutes définies sur le même domaine, à la même échalle et avec la même interprétation pour toutes les entreprises i et tous les instants t.
- **Régularité** : la fonction f_i est supposée suffisamment lisse et ne varie pas brutalement pour des petites variations de t.
- **Colocalisation** : chaque instant t a la même interprétation pour toutes les entreprises. Par exemple, t=48 correspond toujours à la fin du premier jour pour tous les individus.

Ces propriétés nous amènent à considérer chaque ligne du jeu de données non pas comme un vecteur de $\mathbb{R}^D$, mais comme la discrétisation d'une fonction de consommation évoluant dans le temps. Nous adaptons donc une approche de Statistiques Fonctionnelles (FDA). L'analyse reposera sur trois étapes principales :

- Reconstruction des courbes continues par lissage B-splines pénalisé.
- Mise en oeuvre d'une Analyse en Composantes Principales Fonctionnelle (ACP fonctionnelle), afin de résumer la variabilité des courbes et d'identifier les principaux modes de variation des profils de consommation.
- Classification des entreprises par clustering, afin de regrouper les consommateurs présentant des comportements similaires.

```r
#Chargement des données
chemin <- "donnees/smart278co.Rdata"
load(chemin)
table <- smart.278co
df <- as.data.frame(table)

#Nb de lignes
n <- nrow(table)
```

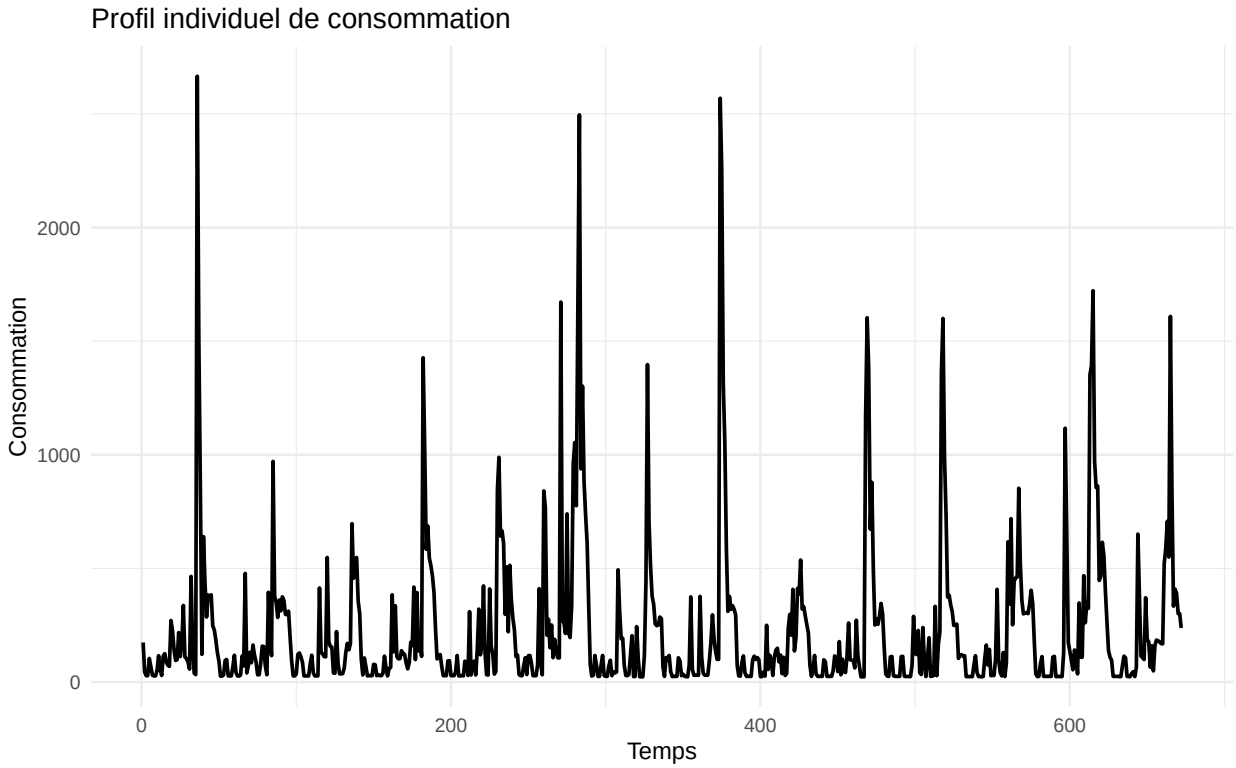
```r
#Nb de cols
p <- ncol(table)
#indices des mesures
time <- 1:p
```

Packages utilisés

- ***fda*** : package permettant la manipulation et création d'objets fonctionnels (fd), le lissage B-splines pénalisé (smooth.basis, fdPar) et l'ACP fonctionnelle (pca.fd).
- ***refund*** : package dédié aux méthodes de regression fonctionnelle, il fournit notamment l'algorithme FACE (fpca.face) qui estime l'opérateur de covariance par lissage direct, très efficace sur grands jeux de données.
- ***mgcv*** : package de modélisation par splines pénalisées, permet d'ajuster des modèles additifs (gam).
- ***lme4*** : package de modèles mixtes, utilisé ici pour ajuster des modèles de Poisson locaux (glmer) dans le cadre de l'approche GFPCA.
- ***cluster*** et ***factoextra*** : ces deux packages sont utilisés pour le clustering. Le premier fournit les algorithmes de classification et le deuxième permet la visualisation des résultats.
- ***ggplot2, tidyr, dplyr*** : packages utilisés pour la manipulation et visualisation des données.

2 - Exploration des données

Avant toute modélisation, on peut représenter des exemples des données brutes :

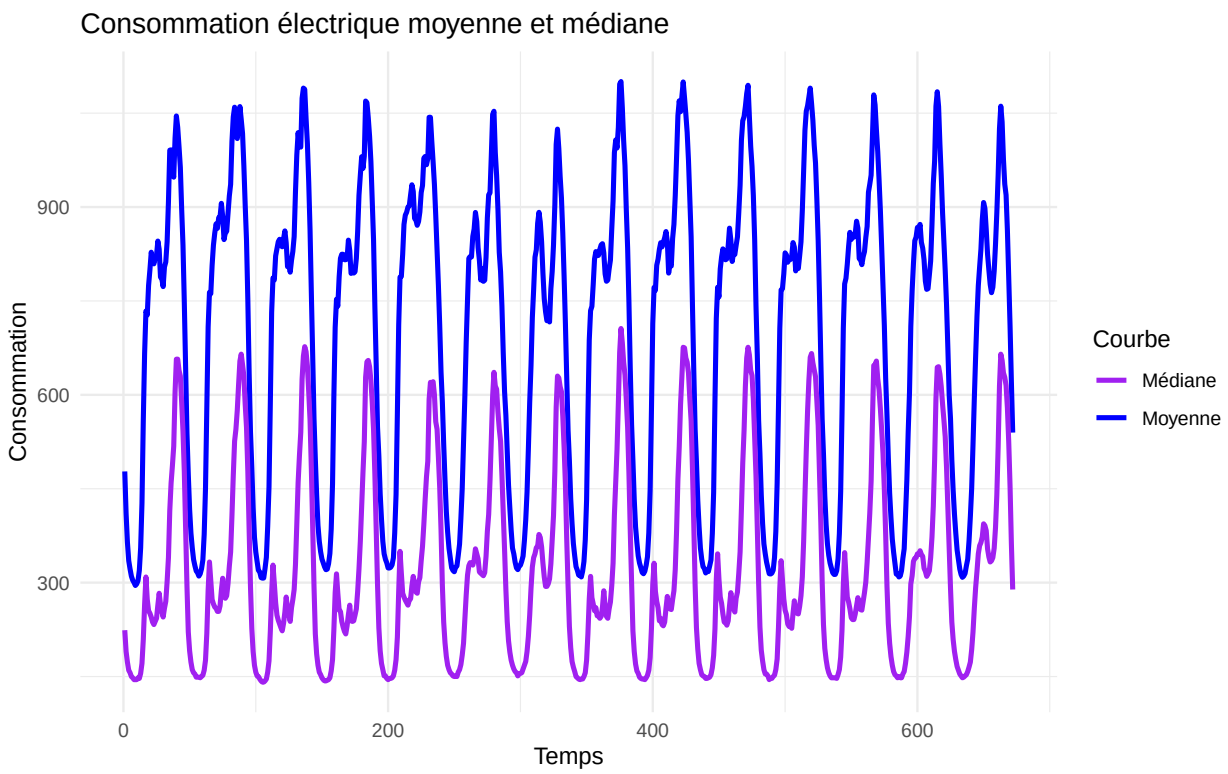


Les profils présentent une structure périodique marquée correspondant aux cycles journaliers. On observe des pics de consommation récurrents, probablement liés aux usages domestiques (matin, soir, weekends), ainsi qu'une variabilité notable entre individus.

Afin de synthétiser cette information, nous calculons ensuite la courbe moyenne et la courbe médiane, plus robuste aux profils atypiques.

Remarque : La médiane calculée ici est une médiane point par point, elle est calculée indépendamment à chaque instant t. Elle ne tient pas compte de la structure globale des courbes. Une alternative plus rigoureuse est la médiane géométrique fonctionnelle, qui peut être calculé grâce au package fda.

```
#Courbe moyenne et mediane
f_mean <- colMeans(table,  na.rm=T)
f_median <- apply(table, 2, median, na.rm=T)
```



La courbe moyenne met clairement en évidence une structure périodique régulière.On observe clairement quatorze cycles journaliers successifs. Chaque cycle présente une dynamique similaire : une phase de faible consommation nocturne suivie de pics plus marqués en journée, probablement associés aux usages domestiques (chauffage, cuisson, éclairage).

La stabilité de la forme moyenne suggère que la variabilité observée entre individus porte principalement sur l'amplitude des pics plutôt que sur la structure temporelle globale.

On observe aussi un écart important entre la courbe moyenne et la courbe médiane, la moyenne est environ 2 à 3 fois plus élevée que la médiane. Cela indique déjà la présence d'entreprises avec une consommation très élevée qui font augmenter la moyenne.

Vérification des valeurs manquantes

```r
sum(is.na(table))
```

```
## [1] 0
```

Aucune valeur manquante n'est détectée dans la matrice de données. Il n'est donc pas nécessaire d'effectuer une étape d'imputation ou de reconstruction préalable.

3 - Lissage semi-paramétrique

L'objectif de cette partie est de passer des observations discrètes Y_i à des courbes continues $\hat{f}_i$, afin de pouvoir appliquer par la suite l'ACP fonctionnelle et le clustering.

Pour réaliser cette transformation, nous utilisons dans les deux cas un lissage par B-splines pénalisées. Les deux approches reposent sur la même pénalisation mais diffèrent dans la méthode de sélection du paramètre de lissage δ : par validation croisée généralisée (GCV) pour la première, et par la méthode REML (Restricted Maximum Likelihood) pour la seconde.

Alternative 1 : Base de B-splines et Validation Croisée

Les données de consommation de chaque entreprise i sont observées aux instants discrétisés $t_1, ..., t_D$, avec D= 672 points qui sont équidistants espacés de 30 minutes sur deux semaines.

L'objectif de cette partie est donc de passer des observations discrètes et bruitées Y_i, aux véritables courbes continues $\hat{f}_i$. En effet, les données brutes comportent du bruit de mesure qui empêche de comparer les individus point par point.

Pour chaque entreprise i, on pose le modèle :

$$Y_i(t_d) = f_i(t_d) + \epsilon_i(t_d), d = 1, ..., D$$

où ϵ_i est le bruit et f_i est estimée dans une base de B-splines $B_1(.), ..., B_q(.)$, de dimension q = K+m, avec :

- K = nombre de noeuds intérieurs équidistants sur [1 : 672]
- m-1 = degré des polynômes par morceaux.

L'estimation de la courbe s'écrit donc :

$$\hat{f}_i(t) = \sum_{k=1}^{q} \hat{\beta}_k B_k(t)$$

Les coefficients $\beta = (\beta_1, ..., \beta_q)^T$ sont estimés en minimisant l'erreur quadratique pénalisée :

$$\sum_{d=1}^{D} \left(Y_i(t_d) - f_i(t_d)\right)^2 \; + \; \delta \int \left(f_i''(t)\right)^2 dt$$

où :

- $\delta \geq 0$ est le paramètre de lissage.
  - Si $\delta \to 0$, la pénalisation est nulle et le courbe risque de sur-ajuster.
  - Si $\delta \to +\infty$, la pénalisation domine et la courbe tend vers une droite, risque de sous-ajustement.
- le terme $\int (f''(t))^2 dt$ pénalise la rugosité de la fonction.

Ainsi, la solution est donnée par :

$$\hat{\beta}_i = (B^T B + \delta P)^{-1} B^T Y_i$$

où B est la matrice D × q dont la d-ième ligne est $(B_1(t_d), ..., B_q(t_d))$, $Y_i = (Y_{i1}, ..., Y_{iD})^T$ le vecteur des observations de l'entreprise i , et P est la matrice de pénalisation telle que $\beta^T P \beta = \int (f_i''(t))^2 dt$. La matrice P se calcule facilement à partir des dérivées secondes des fonctions de base B-splines.

Choix du nombre de noeuds K

La règle de Ruppert et al. (2002) préconise $K = min(0.25 \times D, 35) = min(0.25 \times 672, 35) = min(168.75, 35) = 35$ noeuds, soit q = K + m = 39 fonctions de base. Dans gam(), nous arrondissons et utilisons le paramètre k=40, ce qui correspond à q=40, ce qui est cohérent avec la règle. De plus, le recours à la pénalisation permet d'utiliser un nombre de noeuds généreux sans risque de sur-ajustement, c'est le rôle de δ.

Pour construire notre base B-splines, nous utilisons la fonction **create.bspline.basis()**, du package ***fda***. On lui spécifie :

- rangeval = c(1,p) : l'intervalle de définiton de la base, ici [1,672]
- nbasis = q : le nombre total de fonction de base, q = K+m = 40.
- norder = m : l'ordre des polynômes m=4, correspondant à des splines cubiques (m-1=3).

```
K <- 35 # Nombre de noeuds
degree <- 3 # Degré des polynômes, m-1
m <- 4 # Ordre (degré+1)
q <- K + degree + 1 # nbasis = q = K+m

# Objet basisfd : notre base
base_bspline <- create.bspline.basis(
  rangeval = c(1, p),
  nbasis = q,
  norder = m
)
```

Soit δ le paramètre de lissage utilisé dans la reconstruction d'une courbe Y_i. Il peut être sélectionné en minimisant le critère de validation croisée généralisée (GCV).

Nous calculons donc le δ^* optimal pour chaque individu séparément en balayant une grille de valeurs $[10{-}5, 10^5]$. Une approche individuelle est plus rigoureuse qu'un δ commun car les entreprises peuvent avoir des profils de consommation très différents.

Pour chaque valeur de δ, la fonction **fdPar()** du package ***fda*** crée un objet de paramètres fonctionnels encodant la base B-splines (base_bspline), l'ordre de dérivation utilisé pour la pénalisation (Lfdobj=2, correspondant à la dérivée seconde f_i''), et la valeur du paramètre de lissage δ.

La fonction **smooth.basis()** va prendre en entrée les instants (times = [1:672]), les observations brutes y_i et l'objet fdpPar contenant la base et le paramètre de pénalisation. Elle résout le critère pénalisé et retourne un objet contenant la courbe lisée $\hat{f}_i$ sous forme de coefficients B-splines, ainsi que la valeur du critère GCV. Le δ^* optimal est celui qui minise ce critère.

Deux fonctions sont définies :

- **calcul_gcv()** : calcule le critère GCV pour un individu i et une valeur δ donnée.
- **delta_opt_i()** : retourne le δ^* optimal pour l'individu i en balayant toute la grille.

```
#Grille des paramètre de lissage à tester
delta_grid <- 10^seq(-5, 5, by=0.5)

#Calcul GCV
calcul_gcv <- function(y_i, delta){
  res <- fdPar(base_bspline, Lfdobj = 2, lambda = delta)
  mean(smooth.basis(time, y_i, res)$gcv)
}

#Choisi le delta optimal qui minimise GCV, en balayant la grille
delta_opt_i <- function(i){
  y_i <- as.numeric(smart.278co[i, ])
  gcv_val <- sapply(delta_grid, calcul_gcv, y_i = y_i)
  delta_grid[which.min(gcv_val)]
}
```

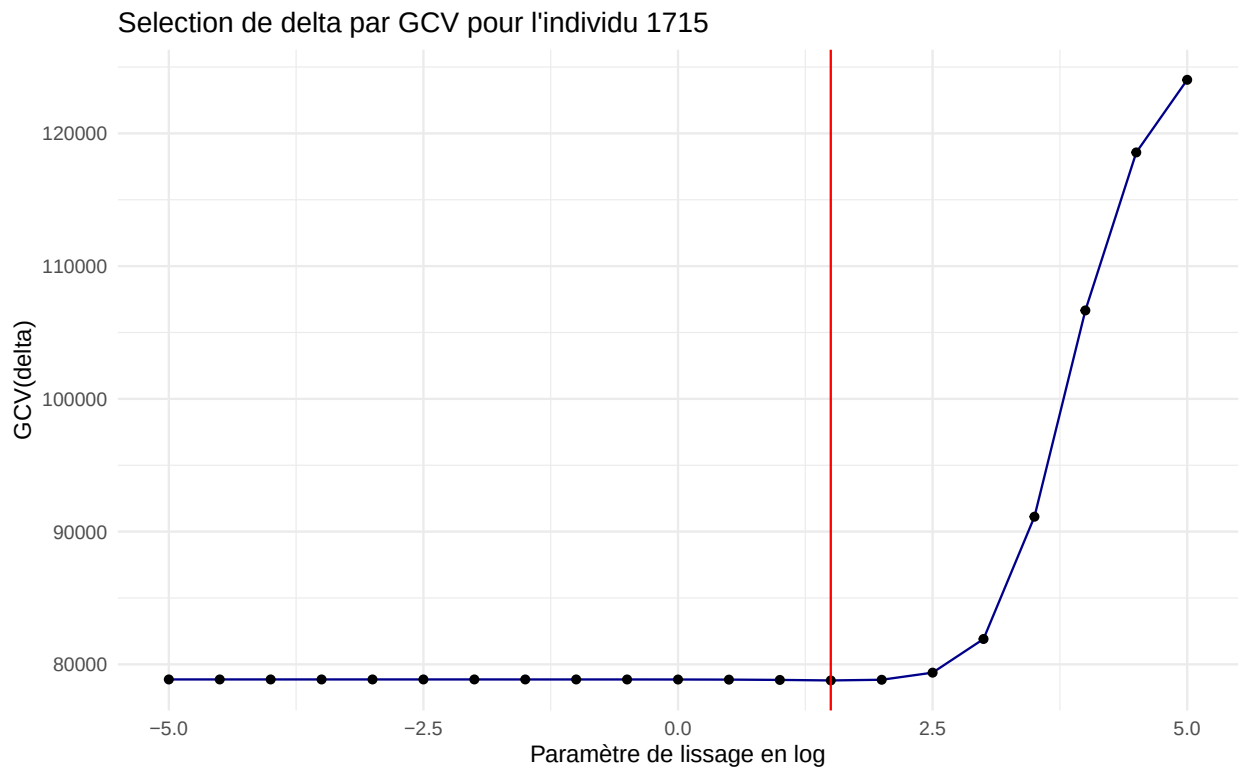
Vérification de l'optimisation sur un individu de test :

```
set.seed(368)
#Sélection aléatoire d'un individu test
index_sample <- sample(1:n, 3)
i <- index_sample[2]

#Extraction des observation de l'individu i
y_i <- as.numeric(smart.278co[i, ])

#Calcul du delta optimal par GCV pour cet individu
delta_opt_i_test <- delta_opt_i(i)
cat("Le delta optimal pour l'individu : ",i," est ", delta_opt_i_test)
```

```
## Le delta optimal pour l'individu :  1715  est  31.62278
```

```
#Calcul du gcv pour chaque valeur de la grille (pour le plot)
gcv_val <- sapply(delta_grid, calcul_gcv, y_i = y_i)

#Dataframe pour Visualisation
df_plot_test1 <- data.frame( delta=delta_grid, gcv = gcv_val)

ggplot( df_plot_test1, aes(x=log10(delta), y=gcv)) +
  geom_line(color="darkblue") +
  geom_point(color="black")+
  #Delta optimal
```

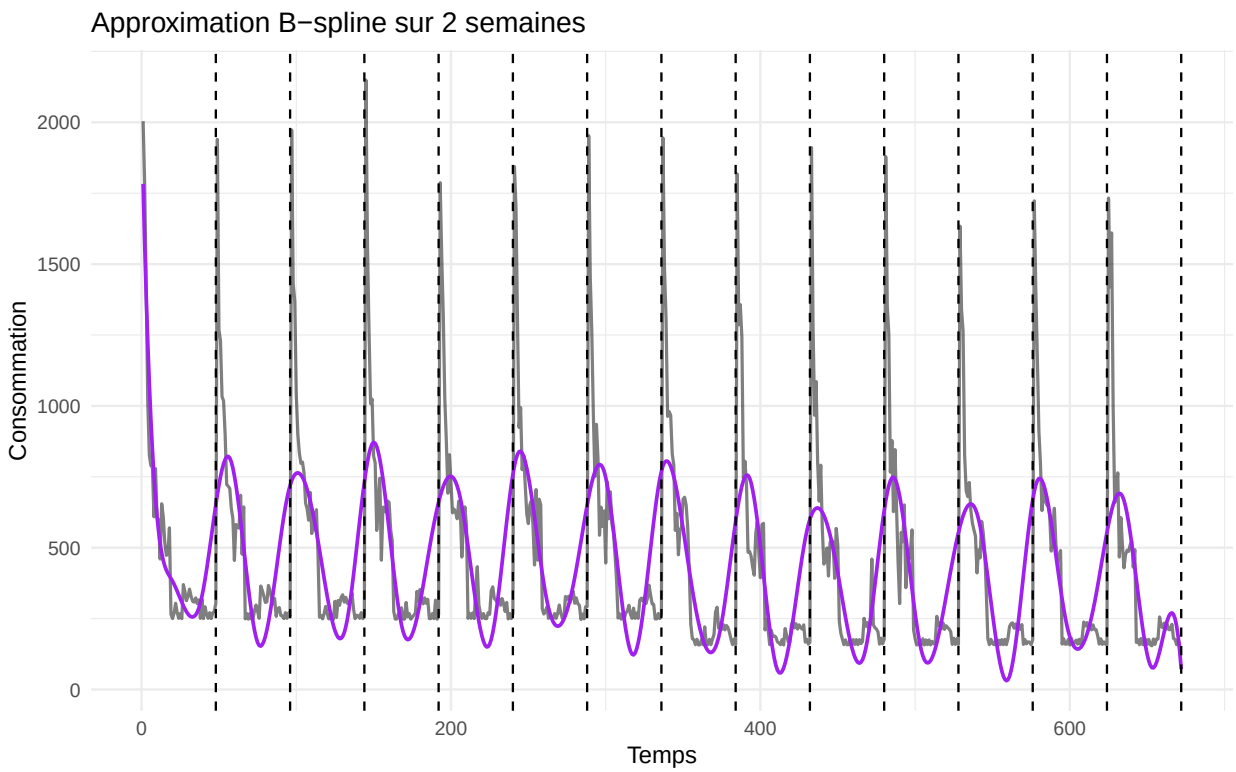
```
  geom_vline(xintercept = log10(delta_opt_i_test), color="red")+
  theme_minimal()+
  labs( title = paste("Selection de delta par GCV pour l'individu", i),
        x="Paramètre de lissage en log",
        y="GCV(delta)")
```



Visualisation de la courbe lissée :

```
#Création de l'objet fdPar avec le delta optimal trouvé
#Lfdobj = 2 : pénalisation dérivée seconde
res_i <- fdPar(base_bspline, Lfdobj = 2, lambda = delta_opt_i_test)

#Lissage de la courbe de l'individu i avec delta_opt
smooth_i <- smooth.basis(time, y_i, res_i)

#Evalusation de la courbe lissée aux instants time
y_lisse <- as.numeric(eval.fd(time, smooth_i$fd))

#Données brutes et courbe lissée dans un dataframe
df_i_fda <- data.frame(temps = time, y = y_i, lisse = y_lisse)

# Plot global sur 2 semaines
#données brutes : gris
#courbe lissée : violet
ggplot(df_i_fda, aes(x = temps)) +
  geom_line(aes(y = y), linewidth = 0.7, alpha = 0.5) +
  geom_line(aes(y = lisse), color = "purple", linewidth = 0.8) +
```

```
  geom_vline(xintercept = seq(48, 672, by = 48), linewidth = 0.5, linetype = "dashed") +
  theme_minimal() +
  labs(title = "Approximation B-spline sur 2 semaines", x = "Temps", y = "Consommation")
```



Nous pouvons maintenant, de la même façon, l'appliquer à l'ensemble des individu (utilisé pour l'approche 2 PCA).

Cependant, le calcul du δ^* pour chaque individu étant couteux en temps de calcul (environ 10-15 min pour les n=6 291 entreprises), les résultats sont calculés une seule fois et sauvegardés dans un fichier .rds. Ils sont ensuite rechargés directement à chaque compilation du rapport.

```
deltas_res <- sapply(1:n, delta_opt_i)
```

Une fois les δ_i^* chargés, la fonction lisser_i() applique le lissage B-splines à chaque individu avec son paramètre optimal. Smooth.basis() retourne un objet fd par individu, c'est à dire n objets séparés. Or pca.fd() nécessite un seul objet fd contenant toutes les courbes.

Nous allons donc extraire les matrices de coefficients B-splines de chaque courbe lissée et les rassembler dans un unique objet fd via la fonction **fd(),** qui encode à la fois les coefficientsβ_i de chaque individu et la base commune. C'est l'objet fd_final qui sera utilisé dans toute la suite de l'analyse.

```
#Lecture des deltas optimaux
deltas_res <- readRDS("donnees/deltas_res.rds")

#Fonction qui va permettre de lisser la courbe associée à l'individu i
lisser_i <- function(i){
  y_i <- as.numeric(smart.278co[i, ])
  res <- fdPar(base_bspline, Lfdobj = 2, lambda = deltas_res[i])
```

```r
  smooth.basis(time, y_i, res)$fd
}

#retourne list de 6291objets fd séparés -> pca.fd a besoin d'un seul objet
fd_list <- lapply(1:n, lisser_i)

#Extraire les coefs
extraire_coefs <- function(fd_i){
  as.numeric(fd_i$coefs)
}

coefs_mat <- sapply(fd_list, extraire_coefs)

#Reconstruire en un seul objet fd
fd_final <- fd(coefs_mat, base_bspline)
```

Alternative 2 : Splines Pénalisées et modèle GAM

La méthode décrite ici provient du livre *Functional Data Analysis with R*. Le modèle est identique à l'Alternative 1. La seule différence réside dans la sélection du paramètre de lissage δ : au lieu d'une recherche manuelle par GCV sur une grille de valeur, celui-ci est estimé automatiquement par la méthode REML (Restricted Maximum Likelihood) intégrée à gam().

On utilise la fonction *gam*() du package **mgcv,** qui ajuste des modèles additifs. Elle prend en entrée un dataframe contenant les observation et une formule spécifiant les termes de lissage. Ici "s(Temps, bs="cr", k=40)", définit un terme de lissage non paramétrique sur la variable 'Temps'."cr", indique que l'on utilise des splines subiques de regréssion, et k=40 fixe la dimension de la base.

La fonction estime simultanément les coefficients $\hat{\beta}$ et le paramètre de lissage δ optimal par REML, et retourne un objet contenant la courbe lissée ainsi que tous les critères d'ajustements.

Exemple sur un individu

```r
# on extrait le premier individu pour la démo
ind_1 <- as.numeric(table[1,])
df_1 <- data.frame(Temps=time, Var=ind_1)

# modèle de lissage
modele_gam <- gam(Var ~ s(Temps, bs = "cr", k = 40),
                  data = df_1,
                  method = "REML")

summary(modele_gam)
```

```
##
## Family: gaussian
## Link function: identity
##
## Formula:
## Var ~ s(Temps, bs = "cr", k = 40)
##
```

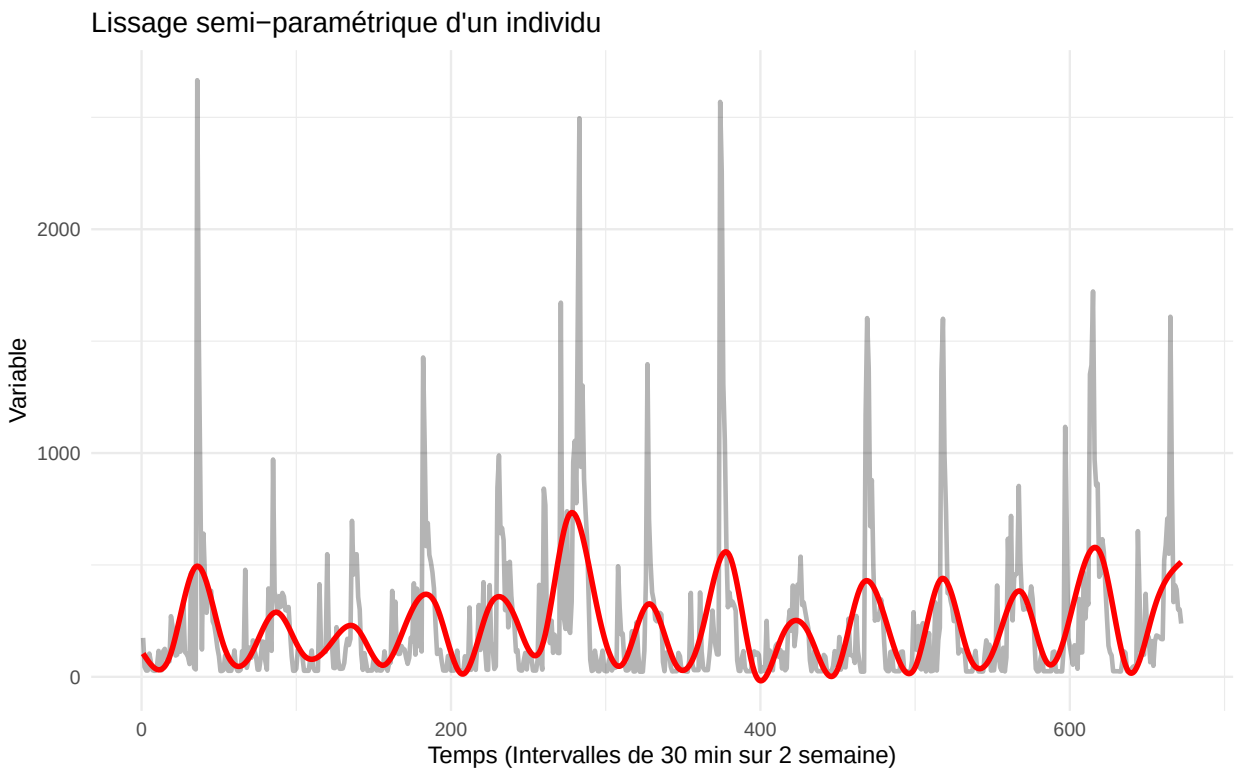
```
## Parametric coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)   228.31      11.01   20.73   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Approximate significance of smooth terms:
##            edf Ref.df     F p-value
## s(Temps) 34.32  37.64 6.624  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## R-sq.(adj) =  0.271   Deviance explained = 30.8%
## -REML = 4818.9  Scale est. = 81510     n = 672
```

Contrairement à la première alternative où le paramètre de lissage optimal devait être recherché manuellement via une grille de valeurs et la validation croisée généralisée (GCV), la fonction $gam()$ automatise ce processus.

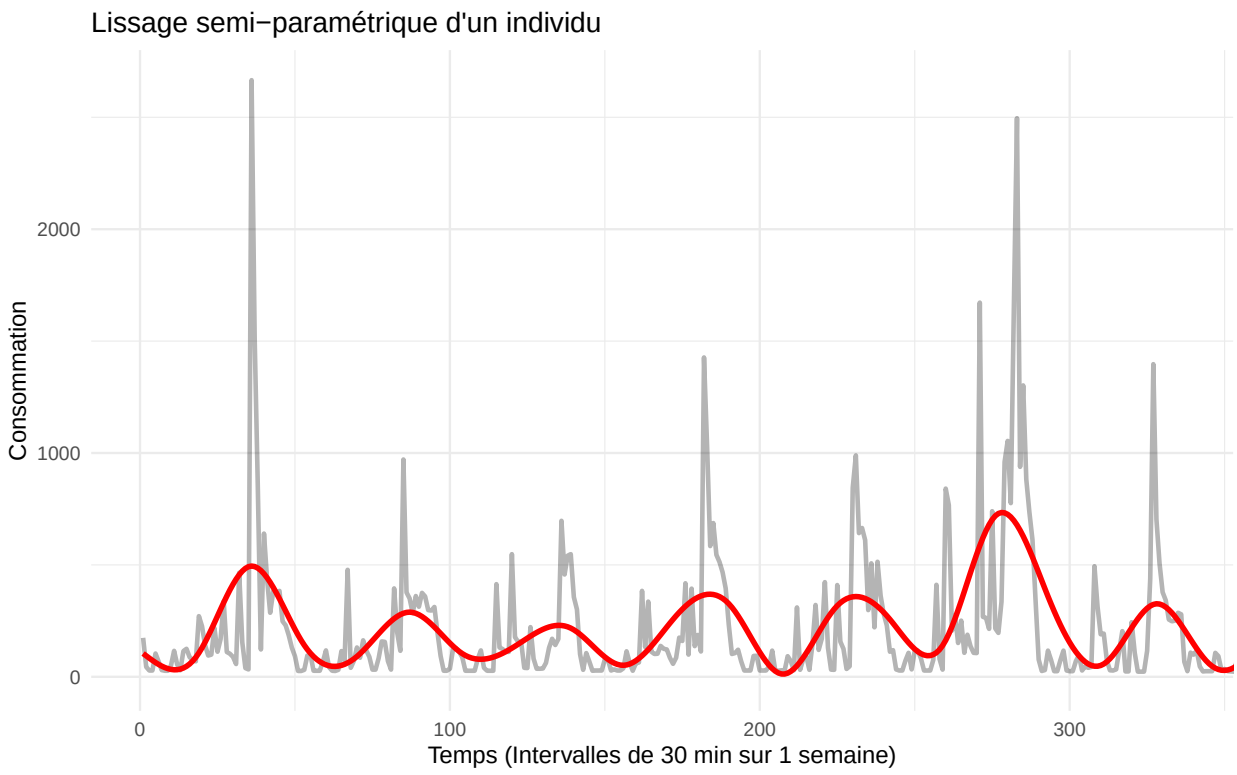
En spécifiant $method =$ "REML", l'algorithme reformule le problème de lissage sous la forme d'un modèle mixte. Le paramètre de pénalité δ est alors estimé directement, comme un ratio entre la variance du signal et la variance du bruit.

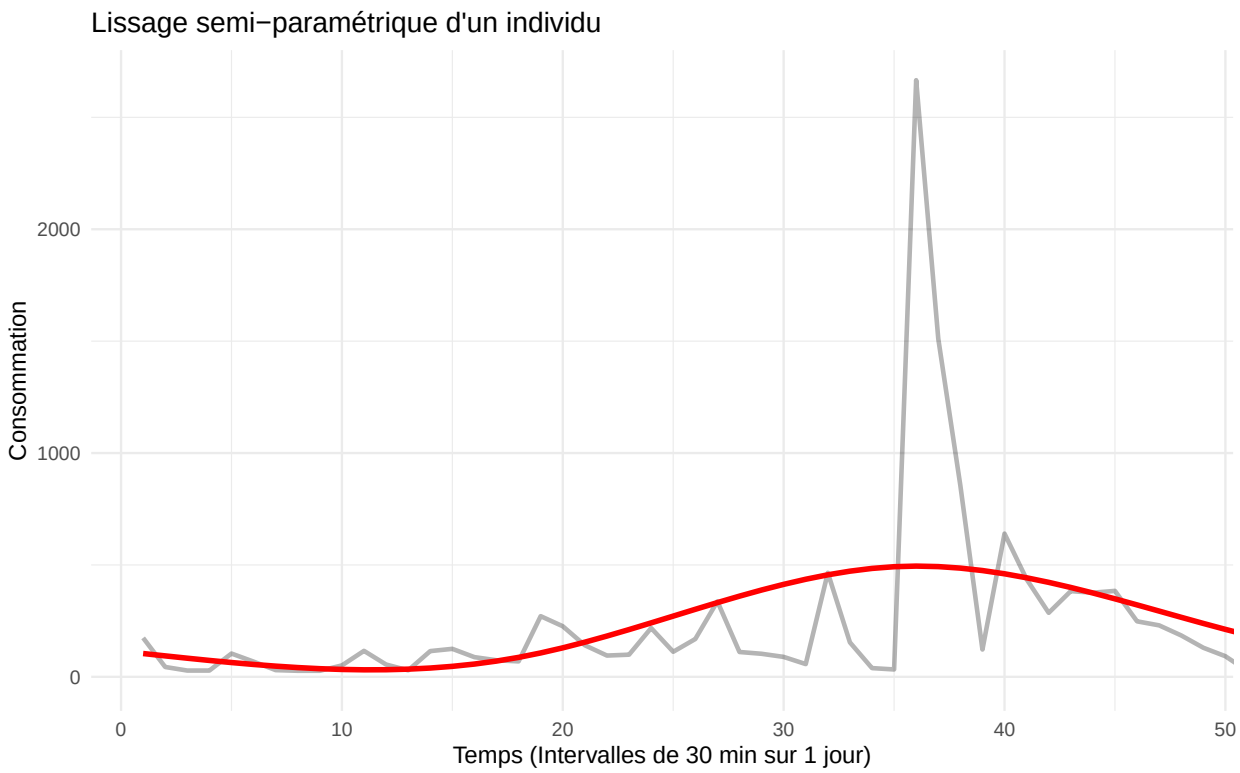
Le degré de lissage optimal est ainsi obtenu en une seule étape, sans risque de sous-lissage.

```r
# prédiction de la courbe lisse
df_1$Courbe_Lisse <- predict(modele_gam, newdata = df_1)

# données brutes VS signal estimé
ggplot(df_1, aes(x = Temps)) +
  geom_line(aes(y = Var), alpha = 0.3, color = "black", size = 1) +
  geom_line(aes(y = Courbe_Lisse), color = "red", linewidth = 1.2) +
  theme_minimal() +
  labs(title = "Lissage semi-paramétrique d'un individu",
       x = "Temps (Intervalles de 30 min sur 2 semaine)",
       y = "Variable")
```




```
# summary(modele_gam)
```





Les grandes vagues dessinées par la courbe rouge représentent le rythme quotidien de consommation du premier individu. On remarque 14 pics et 14 creux. on en déduit que les creux correspondent aux périodes de faible consommation (la nuit) et les pics les périodes de haute consommation (milieu de journée).

Après avoir testé sur un individu et visualiser la qualité du lissage, on applique à l'ensemble des n = 6291 individu.

Application à tous les individus (pour approche 2 PCA)

Lors de l'exploration des données, 16 entreprises ont une consommation constante sur l'ensemble des deux semaines. Parmi elles, 15 affichent une consommation nulle sur les 672 instants, et une seule présente une valeur constante non nulle égale à 9. Ces profils ont donc une variance strictement nulle, ce qui pose problème lors l'estimation du paramètre de lissage δ. En effet, $gam()$ ne peut pas ajuster un modèle sans variabilité.

Nous avons donc décidé d'exclure ces 16 individus de l'analyse, l'effectif final est donc n = 6275.

```
#Individu ayant un consommation nulle
#Renvoie True si la variance de l'individu i = 0 / conso constante
verif_conso <- function (i){
  y_i <- as.numeric(smart.278co[i, ])

  var(y_i)==0 || any(!is.finite(y_i))
}

probleme_conso <- sapply(1:n, verif_conso)
idx_pb <- which(probleme_conso)
idx_ok <- which(!probleme_conso)
cat("Nombre d'ind avec conso constantes :", length(idx_pb),"\n")
```

```
## Nombre d'ind avec conso constantes : 16
```

```r
cat("Nombre d'ind ok :", length(idx_ok))
```

```
## Nombre d'ind ok : 6275
```

```r
#Fonction pour lisser chaque courbe de chaque individu
lisser_gam_i <- function(i){

  y_i <- as.numeric(smart.278co[i, ])

  df_i <- data.frame(Temps = time, Y=y_i)

  mod <- gam(Y~ s(Temps, bs = "cr", k = 40),
                 data = df_i,
                 method = "REML")

  #Recuperer le delta optimal trouvé par REML
  delta_opt <- mod$sp
  res <- fdPar(base_bspline, Lfdobj = 2, lambda = delta_opt)
  smooth.basis(time, y_i, res)$fd
}

#Matrice pxn des courbes lissées
fd_list <- lapply(idx_ok, lisser_gam_i)
coefs   <- sapply(fd_list, function(fd_i) as.numeric(fd_i$coefs))
fd_final <- fd(coefs, base_bspline)
#Ligne executée qu'une fois
saveRDS(fd_final, file = "donnees/fd_final.rds") #15-20 min
```

```r
#chargement du lissage
fd_final <- readRDS("donnees/fd_final.rds")
```

Comparaison des deux approches de lissage

Pour évalué leur qualité respective, on peut calculer l'erreur quadratique moyenne (EQM) sur un individu test par exemple :

$$EQM = \frac{1}{D}\sum_{d=1}^{D}(Y_i(t_d) - \hat{f}_i(t_d))^2$$

```r
#EQM pour REML (y_lisse déjà calculé pour GCV)
y_lisse_reml <- predict(gam(Var ~s(Temps, bs="cr", k=40),
                        method="REML", data=data.frame(Temps=time, Var=y_i)))

#Comparaison
eqm_gcv  <- mean((y_i - y_lisse)^2)
eqm_reml <- mean((y_i - y_lisse_reml)^2)
```

```
## EQM GCV : 70018.25
```

```
## EQM REML : 70773.71
```

```
## Différence : 1.08 %
```

Les résultats obtenus sont très similaires. Nous choisissons donc la méthode REML pour le lissage des individus, notamment à cause de son avantage computationnel.

4 - Analyse en Composantes Principales Fonctionnelle (ACPF)

L'Analyse en Composantes Principales Fonctionnelle (FPCA) permet de résumer la variabilité des courbes $Y_i(t)$ autour de leur moyenne fonctionnelle $\mu(t)$.

Le principe consiste à rechercher des fonctions propres $\phi_k(t)$ maximisant la variance projetée :

$$Y_i(t) = \mu(t) + \sum_{k=1}^{\infty} \xi_{ik}\phi_k(t)$$

où :

- $\mu(t)$ est la courbe moyenne de la population,
- $\phi_k(t)$ sont les fonctions propres,
- $\xi_{ik} = \int (Y_i(t) - \mu(t))\phi_k(t)dt$ sont les scores principaux de l'individu i, obtenus en projetant sa courbe centrée sur la fonction propre ϕ_k. Ils mesurent l'écart de la courbe de l'individu i à la moyenne dans la direction ϕ_k.

Les fonctions propres $\phi_k(t)$ sont obtenues en diagonalisant l'opérateur de covariance : $C(s,t) = \frac{1}{n}\sum_{i=1}^{n}(Y_i(s) - \mu(s))(Y_i(t) - \mu(t)$. Contrairement à l'ACP traditionnelle où c'est une matrice de covariance qui est diagonalisée, ici c'est un opérateur de covariance.

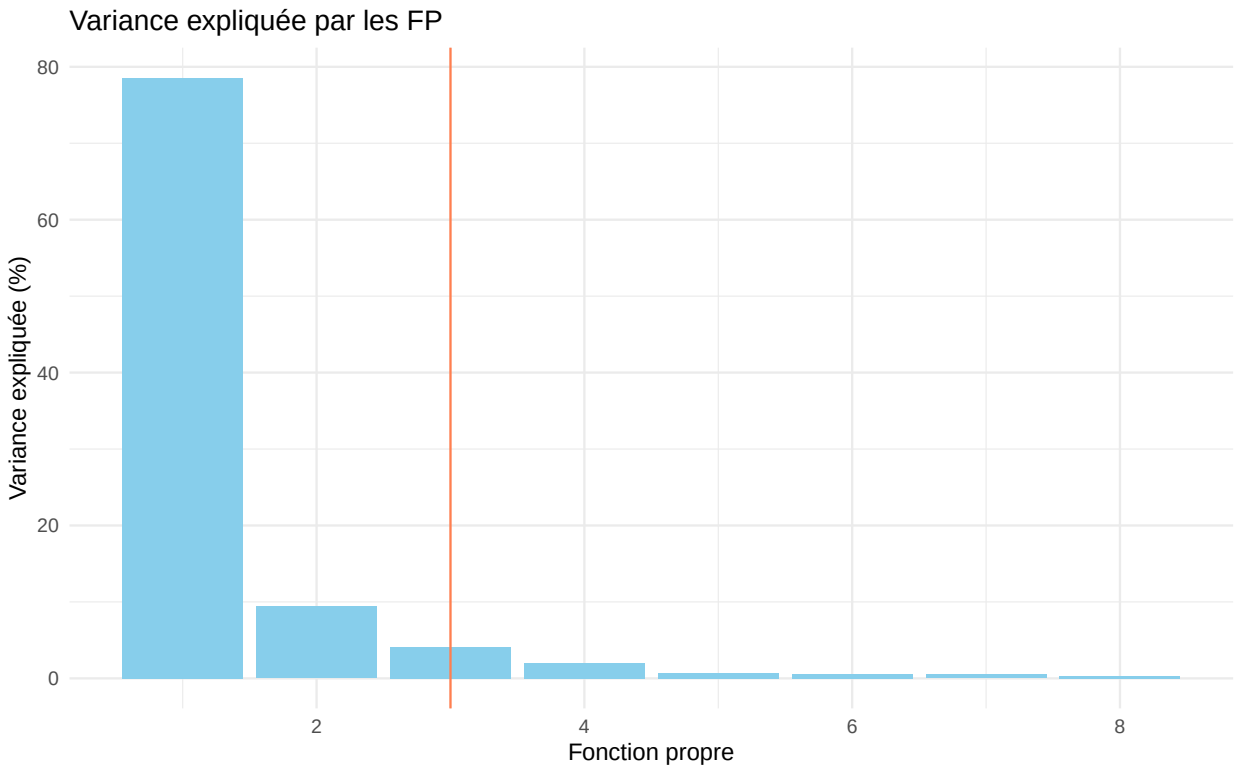
Remarque : Les courbes sont centrées par rapport à la moyenne $\mu(t)$ mais pas réduites. Contrairement à l'ACP traditionnelle sur données multivariées, on ne divise pas par l'écart-type car toutes les mesures sont dans la même unité (la consommation électrique).

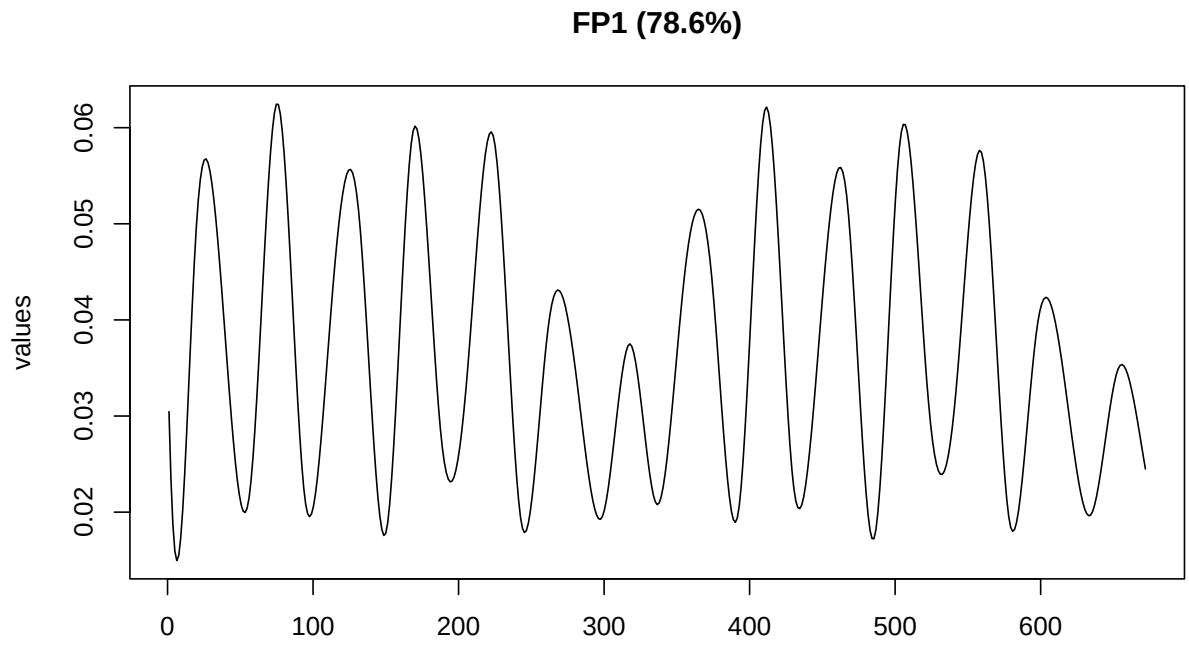
Approche 1 : pca.fd() - package fda

Cette approche consiste à d'abord lisser individuellement les courbes (section précédente), puis à appliquer l'ACPF via la fonction **'pca.fd()' du package 'fda'**. L'objet **'fd' passé en entrée, contient les coefficients B-splines $\hat{\beta}_i$ de chaque courbe lissée $\hat{f}_i$, ainsi que la base de B-splines commune. A partir de ces coefficients, 'pca.fd()'** estime l'opérateur de covariance **C(s,t),** défini récédemment, puis en extrait les fonctions propres ϕ_k et les scores ξ_{ik} par diagonalisation.

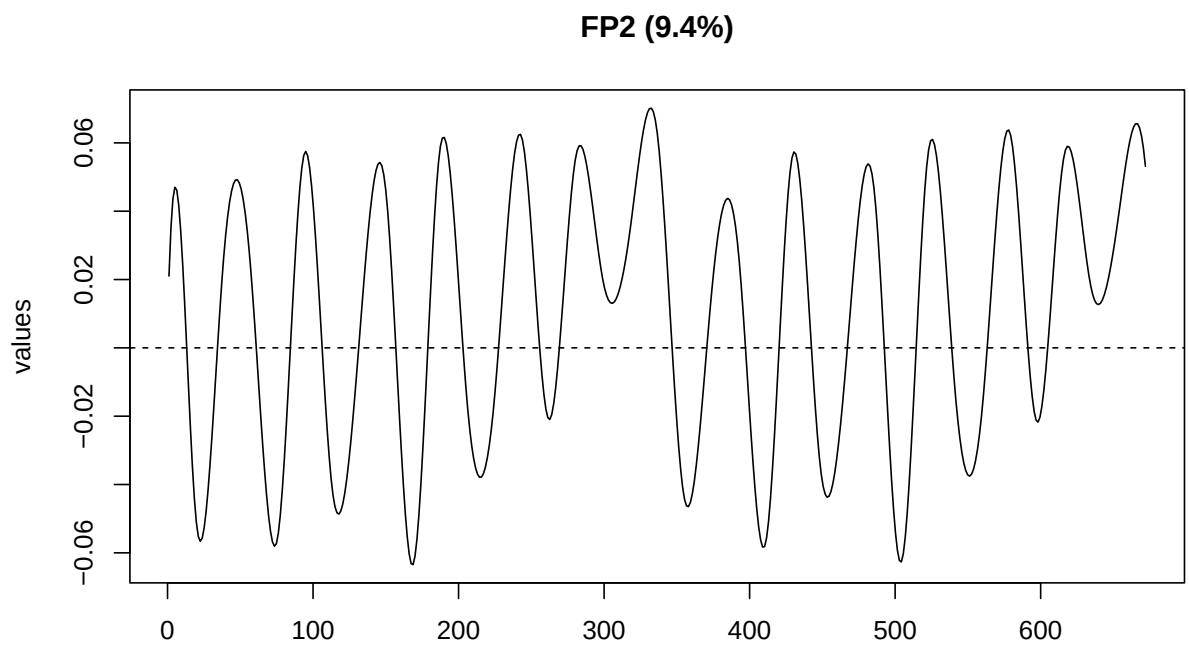
```r
# PCA fonctionnelle
pca_res <- pca.fd(fd_final, nharm = 10)

# Variance expliquée
var <- pca_res$varprop *100
var_cum <- cumsum(pca_res$varprop*100)

data.frame(FP  = 1:10,
           prop = round(var[1:10], 2),
           cum  = round(var_cum[1:10], 2))
```

```
##    FP  prop   cum
## 1   1 78.57 78.57
## 2   2  9.40 87.97
## 3   3  4.09 92.06
## 4   4  2.04 94.09
## 5   5  0.73 94.82
## 6   6  0.55 95.37
## 7   7  0.50 95.87
## 8   8  0.35 96.22
## 9   9  0.32 96.54
## 10 10  0.29 96.83
```

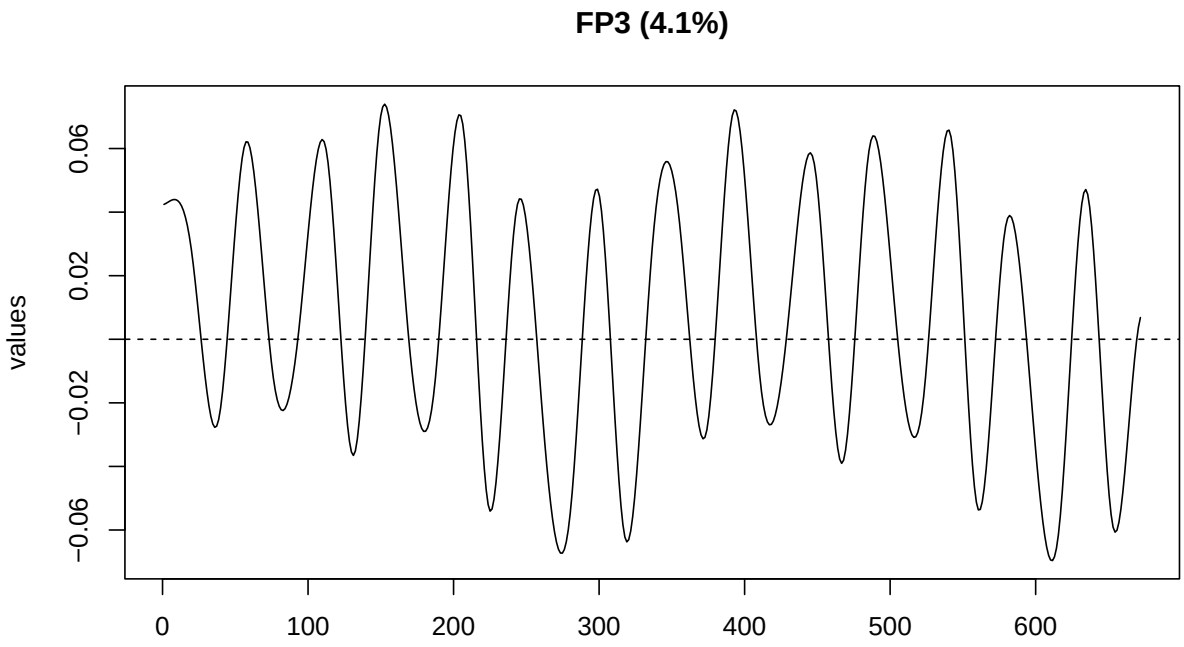



FP1 (78.6%)
values
0.02
0.03
0.04
0.05
0.06
0
100
200
300
400
500
600

```
## [1] "done"
```


FP2 (9.4%)
values
−0.06
−0.02
0.02
0.06
0
100
200
300
400
500
600

```
## [1] "done"
```




```
## [1] "done"
```

Les deux premières fonctions propres :

FP1 - La première fonction propre présente une forme positive et régulière sur l'ensemble des deux semaines, avec une structure périodique journalière visible. Elle explique la majeure partie de la variance totale.

FP2 - La deuxième fonction propre oscille entre le positif et le négatif, traduisant un contraste temporel dans les profils de consommation.

Une interprétation physique détaillée de ces composantes est proposées dans l'Approche 2 (section suivante).

Approche 2 : fpca.face() — package refund

Approche, proposée par Xiao et al. (2016) repose sur une idée différente de l'approche 1 : au lieu de lisser les individus un à un, puis d'estimer l'opérateur de covariance à partir de courbes lissées, on estime et lisse *directement* la matrice de covariance globale C(s,t) à partir des données brutes. Concrètement, la méthode FACE (Fast Covariance Estimation) procède en deux étapes :

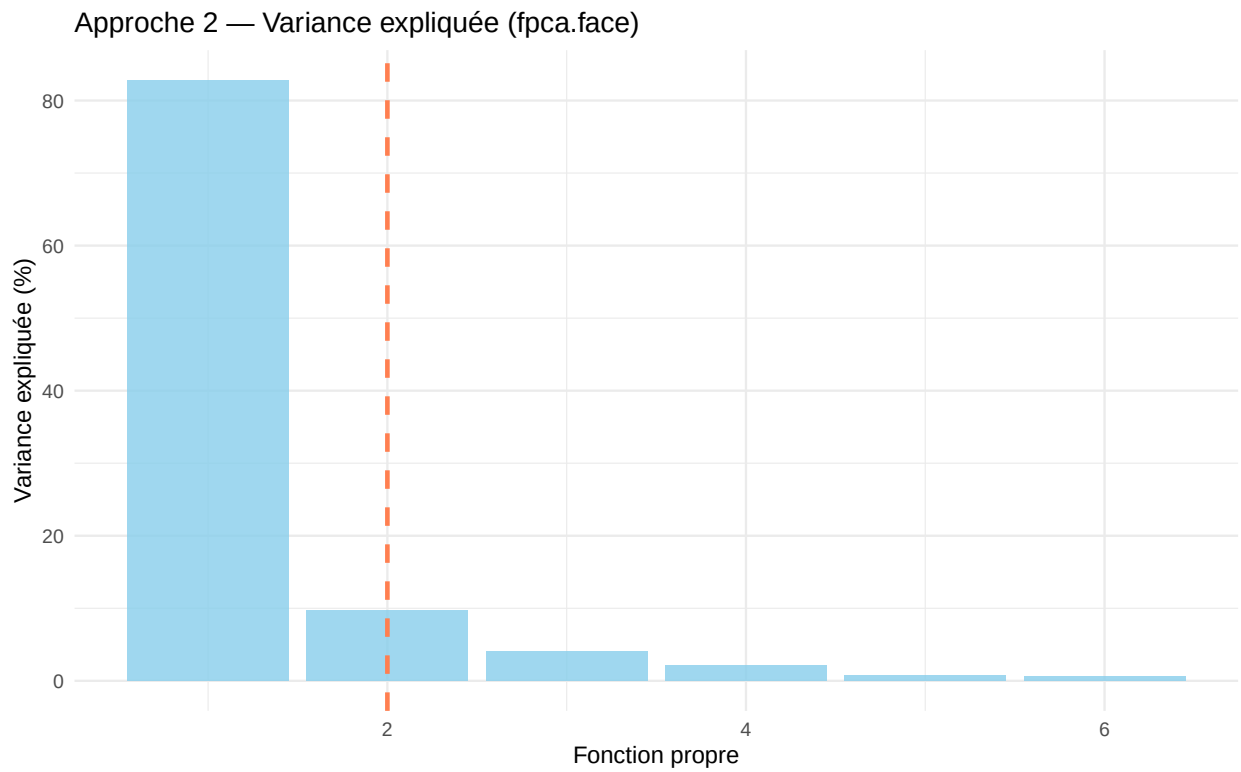
- Elle estime d'abord la matrice de covariance brute à partir des n courbes observées.
- Elle lisse ensuite cette matrice de covariance par splines pénalisées, ce qui permet d'en extraire des fonctions propres ϕ_k et les scores de chaque individu, sans avoir à lisser les individus un à un.

Cette approche est particulièrement pratique pour des grands jeux de données car elle évite n opérations de lissage individuelles.

On utilise donc la fonction $fpca.face()$ du package $refund$ qui implémente cet algorithme .On extraie ici les deux premières composantes (PC1 et PC2), que l'on multiplie par la suite par $\sqrt{p}$. On lui demande 35 noeuds ($knots = 35$) et de retenir les composantes expliquant 95% de la variance ($pve = 0.95$).

Pour la suite de notre analyse, nous isolons spécifiquement les deux premières composantes principales fonctionnelles.

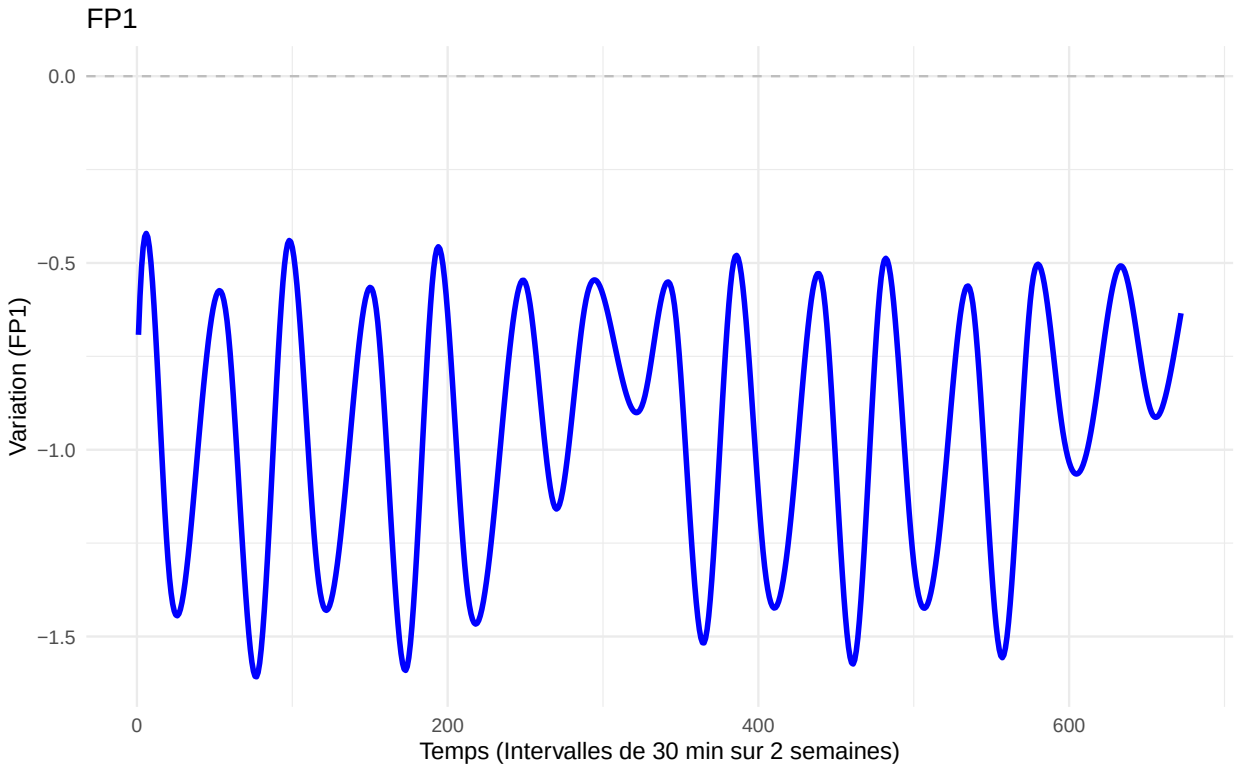
```r
W <- as.matrix(smart.278co[idx_ok, ])

modele_fpca <- fpca.face(Y = W, argvals = time, knots = 35, pve = 0.95)
# summary(modele_fpca)

#on extraie les éléments de la FCPA
courbe_moyenne <- modele_fpca$mu
fonctions_propres <- modele_fpca$efunctions
scores_individuels <- modele_fpca$scores
valeurs_propres <- modele_fpca$evalues

# on extrait les fonctions propres
pc1 <- fonctions_propres[, 1] * sqrt(p)
pc2 <- fonctions_propres[, 2] * sqrt(p)

data_plot <- data.frame(Temps = time, FP1 = pc1)
data_plot2 <- data.frame(Temps = time, FP2 = pc2)


# Variance expliquée
var2     <- modele_fpca$evalues / sum(modele_fpca$evalues) * 100
var_cum2 <- cumsum(var2)
nb_cp2 <- min(which(var_cum2 >= 90))
df_var2 <- data.frame(
  FP   = 1:length(var2),
  prop = var2
)

ggplot(df_var2, aes(x = FP, y = prop)) +
  geom_col(fill = "skyblue", alpha = 0.8) +
  geom_vline(xintercept = nb_cp2, color = "coral", linetype = "dashed", linewidth = 1) +
  theme_minimal() +
  labs(title = "Approche 2 - Variance expliquée (fpca.face)",
       x = "Fonction propre", y = "Variance expliquée (%)")
```




```
df_var2
```

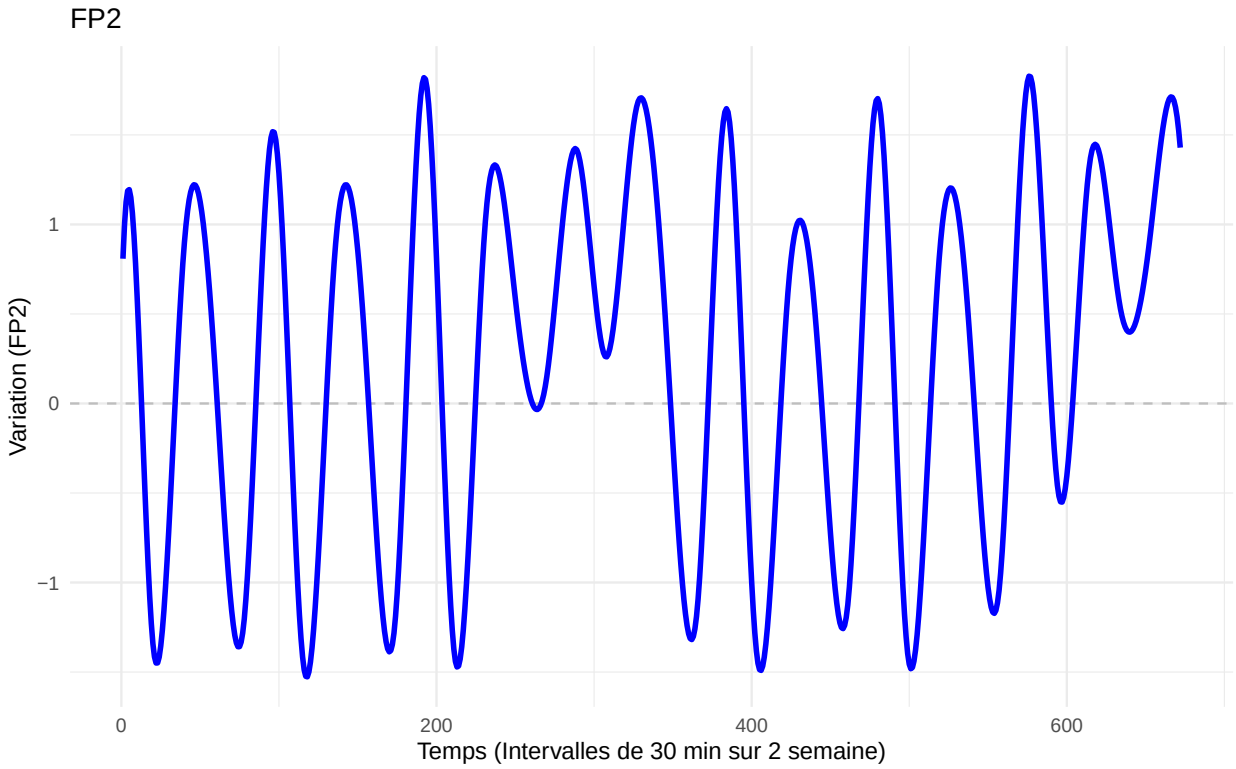
```
##   FP       prop
## 1  1 82.8035098
## 2  2  9.7569668
## 3  3  4.0434672
## 4  4  2.0981162
## 5  5  0.6936477
## 6  6  0.6042922
```

```
# visualisation de la PC1
ggplot(data_plot, aes(x = Temps)) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray") +
  geom_line(aes(y = FP1), color = "blue", linewidth = 1.2) +
  theme_minimal() +
  labs(title = "FP1",
       x = "Temps (Intervalles de 30 min sur 2 semaines)",
       y = "Variation (FP1)")
```



On observe une courbe strictement positive qui présente une strucutre périodique régulière sur les deux semaines. On remarque que le même schéma se répète chaque jours : des pics en journée et des creux pendant la nuit. On distingue également une différence entre les jours de semaine et le week-end. En effet les pics du week-end sont différents en amplitude, montrant un comportement de consommation distincts les jours non travaillés.

```
# visualisation de la PC2
ggplot(data_plot2, aes(x = Temps)) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray") +
  geom_line(aes(y = FP2), color = "blue", linewidth = 1.2) +
  theme_minimal() +
  labs(title = "FP2",
       x = "Temps (Intervalles de 30 min sur 2 semaine)",
       y = "Variation (FP2)")
```



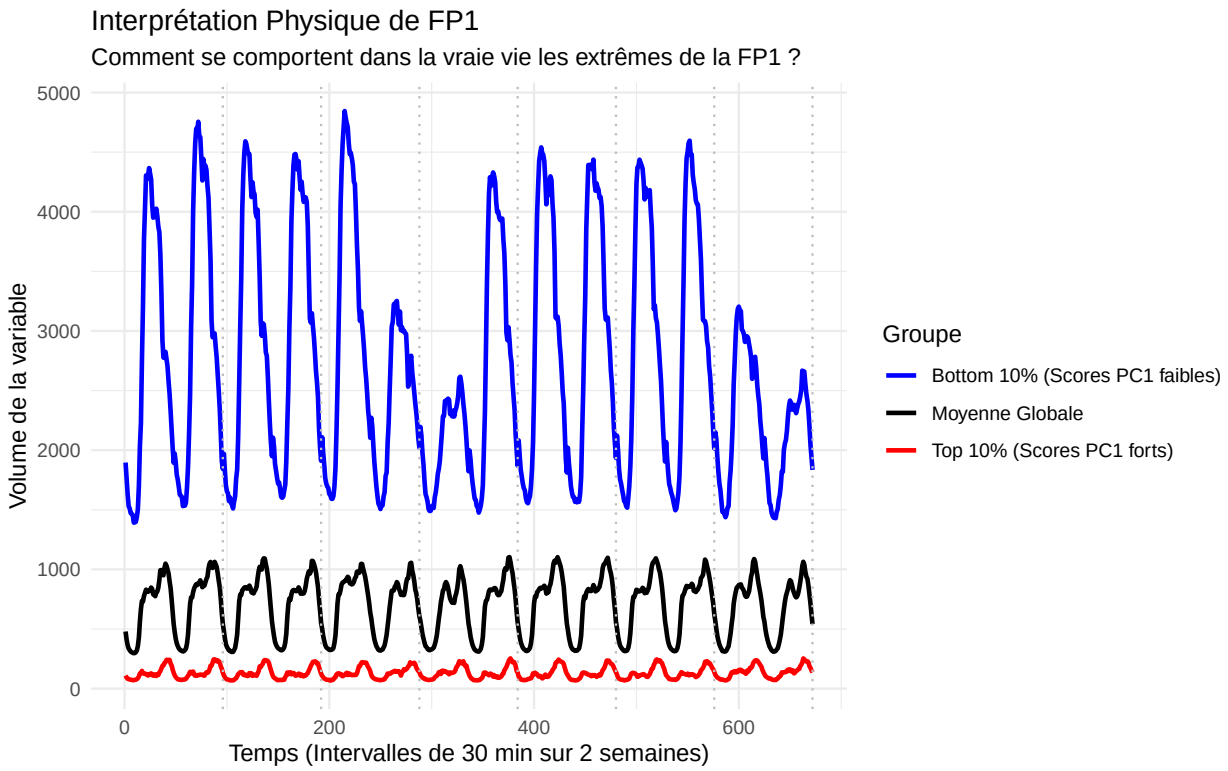
On observe là aussi une courbe qui ondule mais cette fois qui prend des valeurs négatives ou nulles.

Interpréation de la FP1

La courbe de la première composante est assez abstraite. On cherche à comprendre ce que la PC1 mesure physiquement sur notre jeu. Pour essayer de traduire, on isole les comportements extrêmes de la population : les individus dans le top 10% des scores de la PC1 et ceux dans le bottom 10% de la PC1. On calcule ensuite la vraie courbe moyenne de ces deux groupes pour observer comment la PC1 affecte la forme globale de la productio sur une semaine.

```
#score de la composante 1
scores_pc1 <- modele_fpca$scores[, 1]

seuil_haut <- quantile(scores_pc1, 0.90)
seuil_bas <- quantile(scores_pc1, 0.10)

#individus extrêmes
index_haut <- which(scores_pc1 >= seuil_haut)
index_bas <- which(scores_pc1 <= seuil_bas)

#vraies moyennes pour groupes extrêmes
courbe_moyenne_top10 <- colMeans(W[index_haut, ])
courbe_moyenne_bot10 <- colMeans(W[index_bas, ])

#dataframe pour graphique
df_interpretation <- data.frame(
  Temps = rep(time, 3),
```

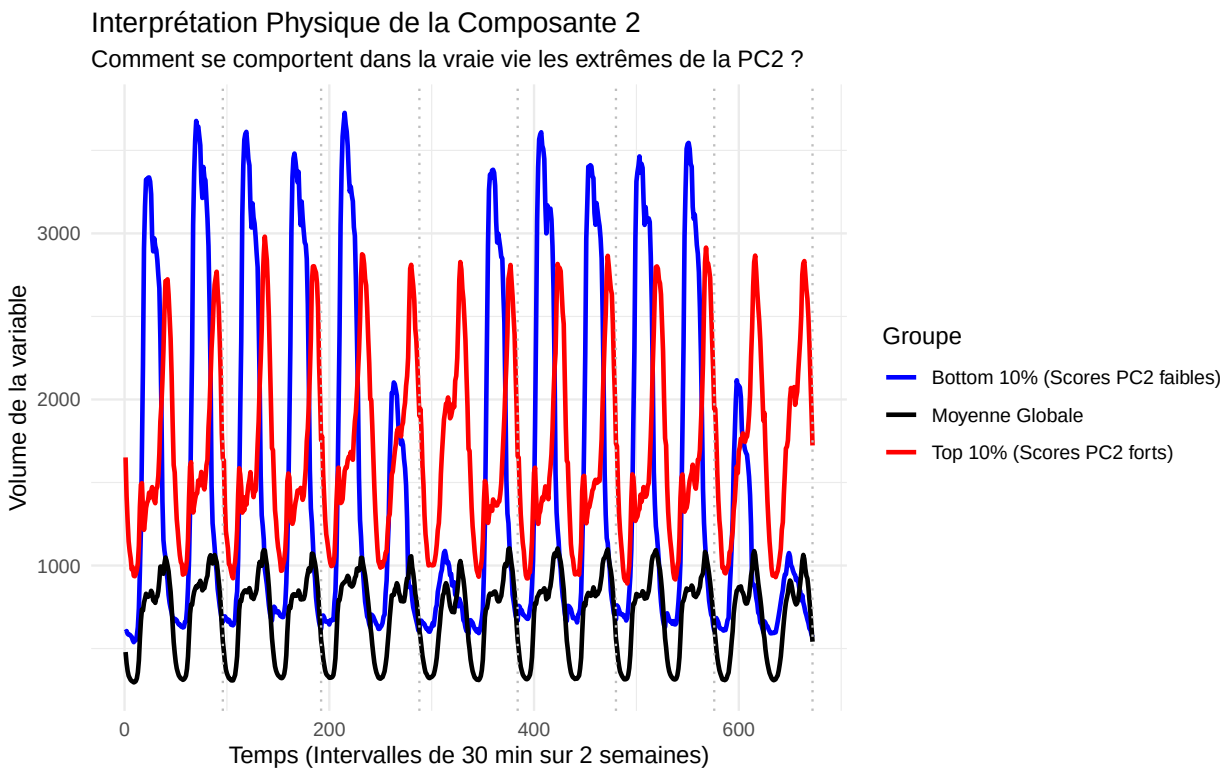
```r
  Activite = c(courbe_moyenne_top10, courbe_moyenne_bot10, courbe_moyenne),
  Groupe = rep(c("Top 10% (Scores PC1 forts)",
                 "Bottom 10% (Scores PC1 faibles)",
                 "Moyenne Globale"), each = 672)
)

ggplot(df_interpretation, aes(x = Temps, y = Activite, color = Groupe)) +
  geom_line(linewidth = 1) +
  scale_color_manual(values = c("Top 10% (Scores PC1 forts)" = "red",
                                "Bottom 10% (Scores PC1 faibles)" = "blue",
                                "Moyenne Globale" = "black")) +
  geom_vline(xintercept = seq(96, 672, by = 96), color = "gray", linetype = "dotted") +
  theme_minimal() +
  labs(title = "Interprétation Physique de FP1",
       x = "Temps (Intervalles de 30 min sur 2 semaines)",
       y = "Volume de la variable",
       subtitle = "Comment se comportent dans la vraie vie les extrêmes de la FP1 ?")
```



On déduit que la première composante, expliquant 82% de la variance totale, agit comme un multiplicateur global sur les deux semaines. On constate que les individus du top 10% consomment massivement plus d'énergie que la moyenne. Leurs pics journaliers sont également bien plus importants. A l'inverse, la consommation du Bottom 10% semble presque plate sur l'échelle du réseau.

Interpréation de la FP2

```r
#score de la composante 1
scores_pc2 <- modele_fpca$scores[, 2]

seuil_haut <- quantile(scores_pc2, 0.90)
seuil_bas <- quantile(scores_pc2, 0.10)

#individus extrêmes
index_haut <- which(scores_pc2 >= seuil_haut)
index_bas <- which(scores_pc2 <= seuil_bas)

#vraies moyennes pour groupes extrêmes
courbe_moyenne_top10 <- colMeans(W[index_haut, ])
courbe_moyenne_bot10 <- colMeans(W[index_bas, ])

#dataframe pour graphique
df_interpretation <- data.frame(
  Temps = rep(time, 3),
  Activite = c(courbe_moyenne_top10, courbe_moyenne_bot10, courbe_moyenne),
  Groupe = rep(c("Top 10% (Scores PC2 forts)",
                 "Bottom 10% (Scores PC2 faibles)",
                 "Moyenne Globale"), each = 672)
)

ggplot(df_interpretation, aes(x = Temps, y = Activite, color = Groupe)) +
  geom_line(linewidth = 1) +
  scale_color_manual(values = c("Top 10% (Scores PC2 forts)" = "red",
                                "Bottom 10% (Scores PC2 faibles)" = "blue",
                                "Moyenne Globale" = "black")) +
  geom_vline(xintercept = seq(96, 672, by = 96), color = "gray", linetype = "dotted") +
  theme_minimal() +
  labs(title = "Interprétation Physique de la Composante 2",
       x = "Temps (Intervalles de 30 min sur 2 semaines)",
       y = "Volume de la variable",
       subtitle = "Comment se comportent dans la vraie vie les extrêmes de la PC2 ?")
```



Cette deuxième fonction propre, expliquant 9.8% de la variance totale, semble capturer une différence dans le cycle journalier entre les entreprises. Celles avec un score ξ_{i2} élevé présentent un pic de consommation journalier plus tardif, tandis que celles avec un score faible ont un pic qui arrive plus tôt dans la journée.

Comparaison des deux approches :

Les deux approches donnent des résultats cohérents : la première composante explique environ 80% de la variance totale dans les deux cas, confirmant que le niveau clobal de consommation est le principale mode de variation entre entreprises. Cependant, on peut noter quelques différences :

L'approche 1 (pca.fd) nécessite 3 composante pour atteindre plus de **90%** de variance expliquée (**92.1**%), contre 2 composantes pour l'approche 2 (fpca.face) (**92.5%**). Les proportions de variance expliquée par composante sont légèrement différentes, ce qui s'explique par le fait qu'elles n'estiment pas l'opérateur de covariance de la même façon.

En termes de temps de calcul, l'approche 2 est nettement plus rapide car elle évite le lissage individuel des n = 6 275 courbes. Elle constitue donc un bon compromis entre précision et efficacité computationnelle.

5 - Clustering

Après réduction de dimension par Analyse en Composantes Principales Fonctionnelle (FPCA), chaque individu est représenté par un vecteur de scores $(\xi_{i1}, \xi_{i2}, \xi_{i3}, \xi_{i4})$, correspondant à sa projection sur les quatre premières fonctions propres. Ces scores résument l'essentiel de la variabilité observée entre les profils de consommation.

Nous utilisons ces coordonnées comme variables d'entrée d'un algorithme de classification non supervisée (K-means), afin d'identifier des groupes de consommateurs présentant des profils fonctionnels similaires.

L'algorithme des K-Means cherche à partitionner les individus en C groupes (clusters) $G_1, \ldots, G_C$ en minimisant l'inertie intra-classe :

$$\min \sum_{c=1}^{C} \sum_{i \in G_c} \|\boldsymbol{\xi}_i - \boldsymbol{\mu}_c\|^2$$

Où $\boldsymbol{\mu}_c$ est le centre de gravité (le profil de scores moyen) du cluster c, et $||.||$ désigne la norme euclidienne standard dans $\mathbb{R}^K$, avec K le nombre de composantes retenues.

Choix du nombre de clusters

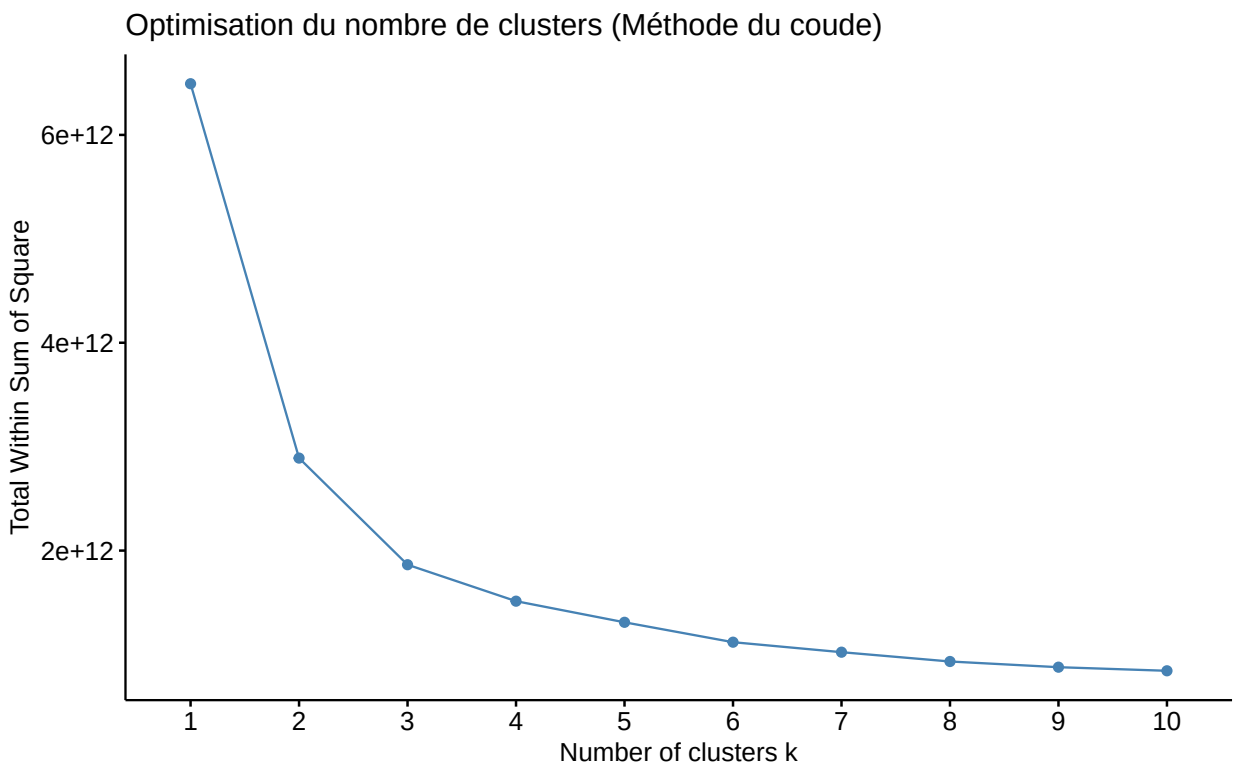
Le nombre de groupes k est déterminé à l'aide de deux critères complémentaires :

- La méthode du coude, qui consiste à analyser la décroissance de l'inertie intra-classe (Within-Cluster Sum of Squares) en fonction de k ;
- L'indice de Silhouette, qui mesure la qualité de séparation entre les clusters en comparant la cohésion intra-classe et la séparation inter-classe.

Ces deux approches permettent d'identifier un compromis entre complexité du modèle et qualité de partition.

```r
# Extraction des scores des 3 premières harmoniques
scores_pca <- modele_fpca$scores[, 1:3]

fviz_nbclust(scores_pca, kmeans, method = "silhouette") +
  labs(title = "Optimisation du nombre de clusters (Silhouette)")
```

```r
# Méthode du coude
fviz_nbclust(scores_pca, kmeans, method = "wss") +
  labs(title = "Optimisation du nombre de clusters (Méthode du coude)")
```



Interprétation

Pour la méthode du coude, la courbe présente une "cassure" très nette à k=2, suivie d'un second point d'inflexion aux alentours de k=4 ou k=5.

Pour l'indice de Silhouette, la courbe décroit légèrement de 0.9 (k=2) à 0.77 (k=4), mais ces valeurs restent élevées, indiquant une bonne séparation des clusters quelle que soit la valeur de k testée (entre 2 et 4).

Décision : Nous retenons **k=4** clusters. Ce choix offre un excellent compromis entre la réduction de l'erreur (inertie) et la simplicité d'interprétation des profils de consommation pour EDF.

Application de l'algorithme K-means

Nous allons ici procéder à un algorithme Kmeans sur les 3 premiers scores FCPA. Nous allons donc extraire les 3 premières colonnes de la matrice des scores de notre modèle FCPA. Nous allons ensuite utiliser la fonction *kmeans*. On utilise aussi $nstart = 25$ pour que l'algo teste plusieurs points de départ et trouve la meilleure séparation possible.

```r
set.seed(123)


clusters <- 4
km_res <- kmeans(scores_pca, centers = clusters, nstart = 25)
```

```r
smart_clusters <- km_res$cluster

# nombre de centrales par groupe
print(table(as.factor(smart_clusters)))
```

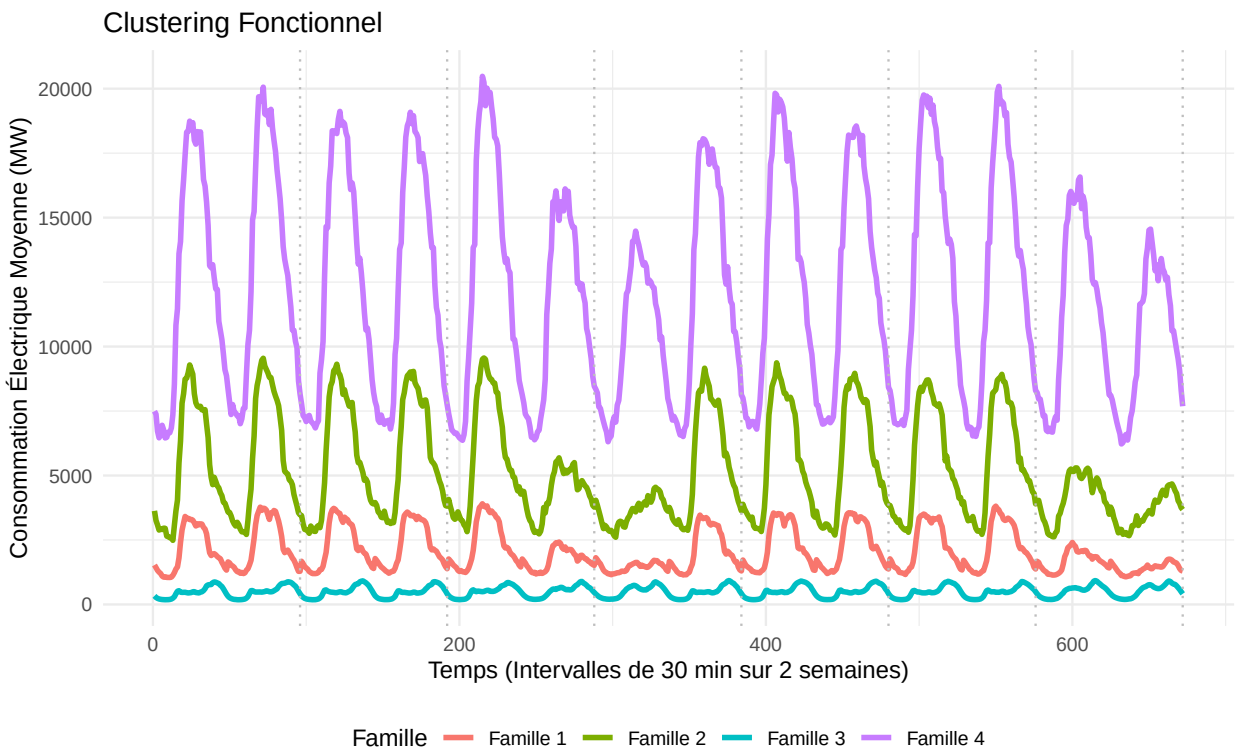
```
##
##    1    2    3    4
##  313  103 5831   28
```

Visualisation et Interprétation des profils types

Maintenant que nous savons quelle centrale appartient au groupe 1,2,3 ou 4, nous retournons dans notre matrice d'origine W et pour chaque groupe, nous calculons la moyenne colonne par colonne (instant par instant) pour tracer la période type de ce groupe.

```r
#matrice qui stocke les profils
courbes_clusters <- matrix(NA, nrow = clusters, ncol = p)

#vraie moyenne par groupe
for(c in 1:clusters) {
  courbes_clusters[c, ] <- colMeans(W[smart_clusters == c, ])
}

df_clusters <- data.frame(
  Temps = rep(1:p, times = clusters),
  Production = as.vector(t(courbes_clusters)),
  Famille = as.factor(rep(paste("Famille", 1:clusters), each = p))
)
```

Nous visualisons maintenant les courbes moyennes par famille :



Interprétation des profils types (centres des clusters)

Sur données brutes

Le **Cluster 1 (n = 5850)** correspond au profil majoritaire de l'échantillon. Il présente une structure journalière régulière, avec des cycles bien définis et relativement stables sur l'ensemble de la période. Ce groupe regroupe donc les foyers présentant un comportement standard avec une faible consommation.

Le **Cluster 2 (n = 28)** rassemble des profils se distinguant par un niveau global plus élevé et une amplitude journalière marquée. Les cycles restent visibles mais présentent une variabilité plus importante au cours du temps, traduisant une intensité énergétique bien supérieure et des fluctuations plus prononcées.

Le **Cluster 3 (n = 310)** correspond à un profil de consommation plus modéré. Les cycles quotidiens sont bien identifiables, mais l'amplitude est globalement plus faible que dans le cluster 2. Ce groupe se situe ainsi à un niveau intermédiaire, avec une dynamique relativement stable mais moins intense.

Enfin, le **Cluster 4 (n = 103)**, très minoritaire, présente une consommation forte et une variabilité plus marquée. Les cycles journaliers demeurent visibles mais l'intensité et l'évolution temporelle diffèrent des autres groupes, suggérant des comportements spécifiques au sein de l'échantillon.

Globalement, la segmentation met principalement en évidence une différenciation selon le niveau global de consommation, tout en révélant des nuances dans l'intensité et la régularité des cycles journaliers.

6 - Pour aller plus loin - GFCPA

Normalité

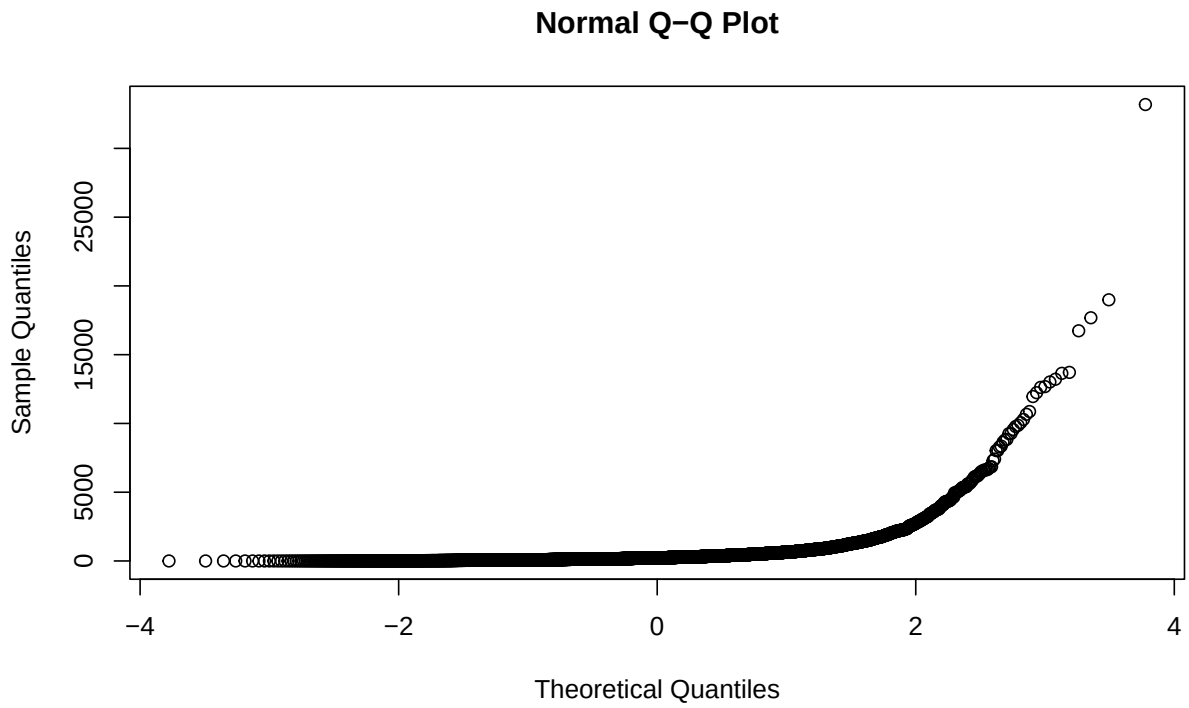
L'Analyse en Composantes Principales Fonctionnelle classique (appliquée précédemment) suppose que les variations autour de la moyenne suivent une loi Normale. Bien que cette hypothèse ne soit pas strictement

nécessaire dans notre contexte, nous explorons ici une approche alternative, la GFPCA, plus adaptée aux données asymétriques, avec des données manquantes par exemple.

Un problème majeur de la FPCA classique est qu'elle peut autoriser mathématiquement des prédictions négatives. Si l'on applique un score très négatif à une petite entreprise, le modèle classique pourrait théoriquement lui prédire une consommation de -15 MW, ce qui est impossible physiquement.

Nous allons vérifier cette hypothèse sur le premier instant temporel :

```
qqnorm(df[,1])
```



On voit que les points ne s'alignent pas tout à fait sur une droite. Ainsi, nos données de consommation ne sont pas gaussiennes sur le premier instant d'où l'utilité possible de la méthode de l'ACPFG (Analyse en Composante Principale Fonctionnelle Généralisée).

Objectif et motivations de la méthode

L'objectif de la GFPCA est d'extraire les profils types de consommation électrique d'un très grand nombre de courbes, en passant par un espace latent (caché), afin de garantir que le lissage mathématique respecte parfaitement la nature physique et probabiliste des données.

Pour respecter la nature de nos données (une consommation étant assimilable à un comptage strictement positif ou nul), nous adoptons le cadre de la Fast Generalized FPCA (FGFPCA). Cette méthode s'appuie sur des Modèles Mixtes Linéaires Généralisés (GLMM) utilisant une loi de Poisson avec un lien logarithmique.

Un modèle mixte est particulièrement adapté ici car il décompose la variance en deux composantes :

Un effet fixe : qui modélise le comportement moyen global du réseau.

Des effets aléatoires : qui estiment la déviation latente spécifique à chaque individu par rapport à la moyenne. Ce sont ces effets aléatoires qui serviront de base à notre nouvelle FPCA.

En utilisant un Modèle Mixte de Poisson avec un lien logarithmique, la GFPCA garantit que la prédiction sera construite sur l'exponentielle du signal latent, et donc strictement positive ou nulle. Ce modèle s'écrit mathématiquement comme suit :

Pour la fenêtre temporelle j, on modélise la production Y_{ij} de la centrale i par :

$Y_{ij} \sim \mathcal{P}(\mu_{ij})$

$\log(\mu_{ij}) = \beta_0 + b_{ij}$

Où μ_{ij} est la consommation moyenne théorique de la centrale i durant la fenêtre temporelle j, β_0 est l'intercepte fixe (la moyenne globale) et b_{ij} est l'effet aléatoire (la déviation latente propre à la centrale i).

Etapes de la Fast GFCPA

Étape 1 : Préparation et binning (fenêtrage)

L'implémentation de cette modélisation rigoureuse impose deux ajustements informatiques majeurs avant de lancer les calculs :

Contrairement à $fpca.face()$ qui accepte une matrice de données globale, les algorithmes de modèles mixtes (comme la fonction $glmer()$ que nous utiliserons en pratique) exigent des données au format long. Nous convertissons donc notre dataframe pour obtenir une seule ligne par observation temporelle et par individu, soit 6 291 individus $\times$ 672 points, c'est-à-dire plus de 4,2 millions de lignes.

Résoudre des modèles mixtes de Poisson sur 4,2 millions de lignes pour chacun des 672 points temporels nécessite une puissance de calcul colossale qui ferait échouer l'algorithme. Pour contourner cette contrainte informatique, nous appliquons la méthode du binning : nous regroupons le temps par fenêtres locales (par exemple ici des blocs de 4 heures, soit 8 intervalles de 30 minutes).

L'objectif est d'estimer rapidement le signal latent de chaque entreprise sur ces blocs réduits, avant d'appliquer un lissage global continu à l'étape suivante.

```r
p <- 672
n <- nrow(W)

df_long <- data.frame(
  ID = as.factor(rep(1:n, times = p)),
  Temps = rep(1:p, each = n),
  Production = as.vector(W)
)

# binning
taille_fenetre <- 8

df_long <- df_long %>%
  mutate(Fenetre = floor((Temps - 1) / taille_fenetre) * taille_fenetre + (taille_fenetre / 2))

centres_fenetres <- unique(df_long$Fenetre)
```

Etape 2 : Estimation latente via modèles mixtes locaux

Pour chaque bloc de 4 heures on isole les données et on ajuste un modèle mixte grâce à la fonction $glmer()$ (issue du package $lme4$). Le modèle sépare l'effet fixe (la consommation moyenne de tous les individus

durant ces 4 heures) de l'effet aléatoire (le décalage propre de chaqur individu). Cet effet aléatoire est notre signal latent, ce qui nous permet de compresser nos données brutes bruitées en une petite matrice d'effets aléatoires.

Puisque nous traitons des données semblables à des données de "comptage", nous utilisons la loi de Poisson avec un lien logarithmique ($family = poisson(link = "log")$). La valeur extraite pour chaque centrale est son effet aléatoire i.e son écart latent par rapport à la moyenne à la moyenne du réseau durant ces 4 heures.

Pour extraire les effets aléatoires de chaque modèle on utilise la fonction $ranef()$.

```
# matrice qui stocke les effets aléatoires
matrice_effets_locaux <- matrix(NA, nrow = n, ncol = length(centres_fenetres))

for (j in 1:length(centres_fenetres)) {
  fenetre_actuelle <- centres_fenetres[j]
  data_locale <- subset(df_long, Fenetre == fenetre_actuelle)

  modele_local <- glmer(Production ~ 1 + (1 | ID),
                        family = poisson(link = "log"),
                        data = data_locale)

  matrice_effets_locaux[, j] <- ranef(modele_local)$ID[[1]]
}
```

Etape 3 : Application de la FCPA sur l'espace latent

Nous prenons la matrice des signaux latents obtenus à la précédente étape et on lui applique une FCPA classique via la fonction $fpca.face$. En effet, puisque l'étape 2 a été calculée sur des blocs de 4 heures, si on s'arrêtait là, la courbe ferait des sauts en escalier. Appliquer une FPCA va permettre de lier ces fenêtres entre elles et d'extraire les fonctions propres ϕ_k de notre réseau.

```
# fcpa classique aux signaux latents
fpca_latente <- fpca.face(Y = matrice_effets_locaux, pve = 0.95)
```

Grâce à cela, nous pouvons extraire les composantes ainsi que les taux de variance qu'elles expliquent. On n'extraie ici que celles qui expliquent au moins 95% de variance.

```
fpca_pour_plot <- fpca.face(Y = matrice_effets_locaux, npc = 10)

valeurs_propres <- fpca_pour_plot$evalues

# Calcul des pourcentages par rapport à la variance totale
variance_expliquee <- (valeurs_propres / sum(valeurs_propres)) * 100
variance_cumulee <- cumsum(variance_expliquee)

#index de la composante qui fait dépasser les 95% de variance expliquée
nb_pc_95 <- which(variance_cumulee >= 95)[1]

#on ne garde que les composantes qui expliquent au moins 95% de variance
df_variance <- data.frame(
  Composante = as.factor(1:nb_pc_95),
  Variance = variance_expliquee[1:nb_pc_95],
  Variance_Cumulee = variance_cumulee[1:nb_pc_95]
)
```

```r
print(df_variance)
```

```
##   Composante  Variance Variance_Cumulee
## 1          1 94.314357         94.31436
## 2          2  2.779747         97.09410
```

On remarque que cette méthode permet d'expliquer bien plus d'information via les deux premières composantes (98%) que les deux alternatives de la FPCA classique.

Etape 4 : Interpolation et Modèle Additif Global

En soit, nous aurions pu nous arrêter à l'étape 3 en extrayant les scores et les fonctions propres du modèle FPCA latent. Mais cette quatrième étape est très importante dans le sens où elle permet de redessiner les courbes, de vérifier visuellement le résultat (i.e que les scores captent bien les tendances).

On réinterpole les courbes ϕ_k de l'étape 3 pour qu'elles aient de nouveau 672 points. Pour cela, nous utilisons la fonction *approx*() qui réalise une interpolation temporelle linéaire pour combler les trous et étirer ces courbes sur la grille initiale

```r
# nombre total de composantes
K <- ncol(fpca_latente$efunctions)

#interpolation des composantes pour repasser aux fenêtres de 30mn
fonctions_propres_interpolees <- matrix(NA, nrow = p, ncol = K)
for (k in 1:K) {
  fonctions_propres_interpolees[, k] <- approx(x = centres_fenetres,
                                               y = fpca_latente$efunctions[, k],
                                               xout = 1:p)$y
}

for(k in 1:K) {
  df_long[[paste0("phi_", k)]] <- fonctions_propres_interpolees[df_long$Temps, k]
}
```

D'après le chapitre 3.2 de **Functional Data Analysis**, le binning a tendance à trop lisser les différences entre les individus. Pour corriger cela, nous lançons un grand modèle additif final (via la fonction *bam*(), une version de *gam*() pour les grands jeux de données) défini par :

$$\log(\mu_i(t)) = f(t) + \xi_{i1}\phi_1(t) + \xi_{i2}\phi_2(t)$$

Où $f(t)$ est la fonction lisse moyenne globale, et ξ_{i1}, ξ_{i2} sont les scores ré-estimés de manière optimale pour chaque centrale.

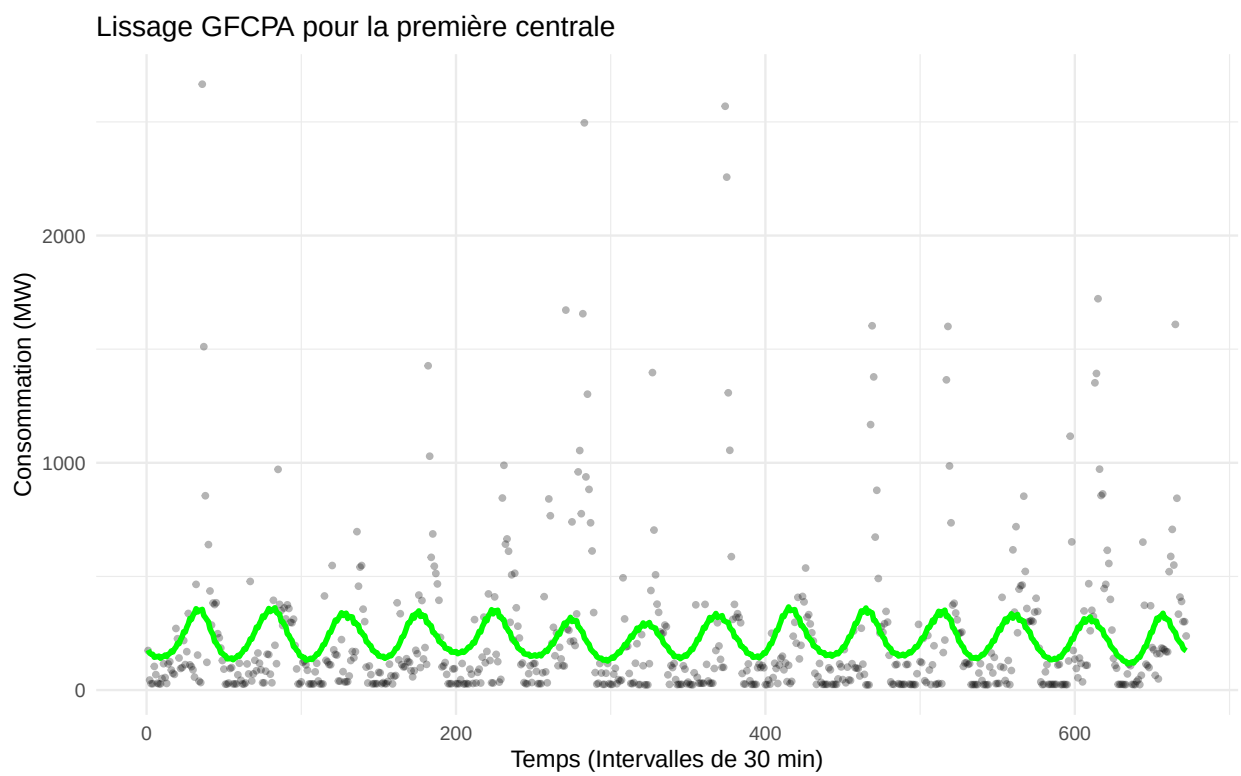
Ce grand modèle *bam* permet en interne de réajuster finement les scores de chaque centrale pour qu'ils soient parfaits.

```r
#on echantillonne pour que ça prenne moins de temps à compiler
df_long_echantillon <- subset(df_long, as.numeric(ID) <= 500)

for (k in 1:2) { # On boucle sur les 2 composantes
  df_long_echantillon[[paste0("phi_", k)]] <- approx(x = centres_fenetres,
                                                     y = fpca_latente$efunctions[, k],
```

```r
                                                    xout = 1:p,
                                                    rule = 2)$y
}



modele_final <- bam(Production ~ s(Temps, bs = "cc", k = 40) +
                      s(ID, by = phi_1, bs = "re") +
                      s(ID, by = phi_2, bs = "re"),
                    method = "fREML", discrete = TRUE, data = df_long_echantillon,
                    family = poisson(link = "log"))

#prédiction lissée
df_long_echantillon$Production_Lissee <- predict(modele_final,
                                                 newdata = df_long_echantillon, type = "response")
```

On regarde la courbe lissée par Fast GFCPA sur le premier individu :

```r
# On isole la centrale n°1 pour comparer les données brutes et le lissage GFPCA
centrale_1 <- subset(df_long_echantillon, ID == 1)

ggplot(centrale_1, aes(x = Temps)) +
  geom_point(aes(y = Production), alpha = 0.3, color = "black", size = 1) +
  geom_line(aes(y = Production_Lissee), color = "green", linewidth = 1.2) +
  theme_minimal() +
  labs(title = "Lissage GFCPA pour la première centrale",
       x = "Temps (Intervalles de 30 min)",
       y = "Consommation (MW)")
```



Nous voyons que la courbe verte suit bien les tendances du nuage de points, ce qui démontre le bon fonctionnement de notre modèle.

7 - Conclusion

Ce projet avait pour objectif d'analyser des profils de consommation électrique à partir d'enregistrements demi-horaires observés sur une période de 14 jours.

Nous avons pu comparer deux approches de lissage (GCV et REML) ainsi que deux approches d'ACPF (*pca.fd* et *fpca.face*). Ces comparaisons nous ont permis de mieux comprendre les avantages et limites de chaque méthode. Finalement l'approche avec fpca.face s'est rélvélée la plus rapide et tout aussi performante que pca.fd.

L'Analyse en Composantes Principales Fonctionnelle a donc permis de résumer l'information contenue dans les 672 observations par individu en un nombre réduit de composantes.La première fonction propre explique plus de 80% de la variance totale, traduisant principalement une variation du niveau global de consommation. Les fonctions propres suivantes capturent des modulations temporelles plus fines de moindre importance, comme le contraste jour/nuit.

La classification K-means réalisée sur les scores des trois premières fonctions propres a mis en évidence quatre profils distincts de consommation, principalement différenciés par leur niveau global de consommation.

Ainsi, l'approche FPCA suivie d'un clustering sur les scores constitue une méthode efficace pour synthétiser et segmenter des données de consommation énergétique massives.

Nous avons également exploré la GFPCA, une approche plus robuste à la non-normalité des données, qui confirme les résultats obtenus avec la FPCA classique.

Plusieurs axes pourraient prolonger ce travail. Le premier serait de la prédiction fonctionnelle : utiliser les profils de la première semaine pour prédire la seconde, via des modèles de régression fonctionnelle. Un second axe pourrait être la détection d'anomalies, c'est à dire identifier automatiquement les entreprises dont le profil s'écarte significativement de leur cluster, ce qui pourrait être utile pour détecter des comportements atypiques ou des pannes.

Références

- Goga Camélia — *Cours de Statistiques Fonctionnelles*, Université de Besançon.
- Ciprian M. Crainiceanu, Jeff Goldsmith, Andrew Leroux, Erjia Cui — *Functional Data Analysis with R*
- Julien Jacques, Cristian Preda — *Functional Data clustering : a survey*
- Jean-Marc Lasgouttes, Analyse fonctionnelle de données, Inria Paris. in (https://who.rocq.inria.fr/Jean-Marc.Lasgouttes/mastere-esd/fda/)